



Adaptive on-device deep-learning for audio and visual sensors at the **extreme-edge**

Manuele Rusci, MSCA Post-Doc, KU Leuven, BE

manuele.rusci@esat.kuleuven.be

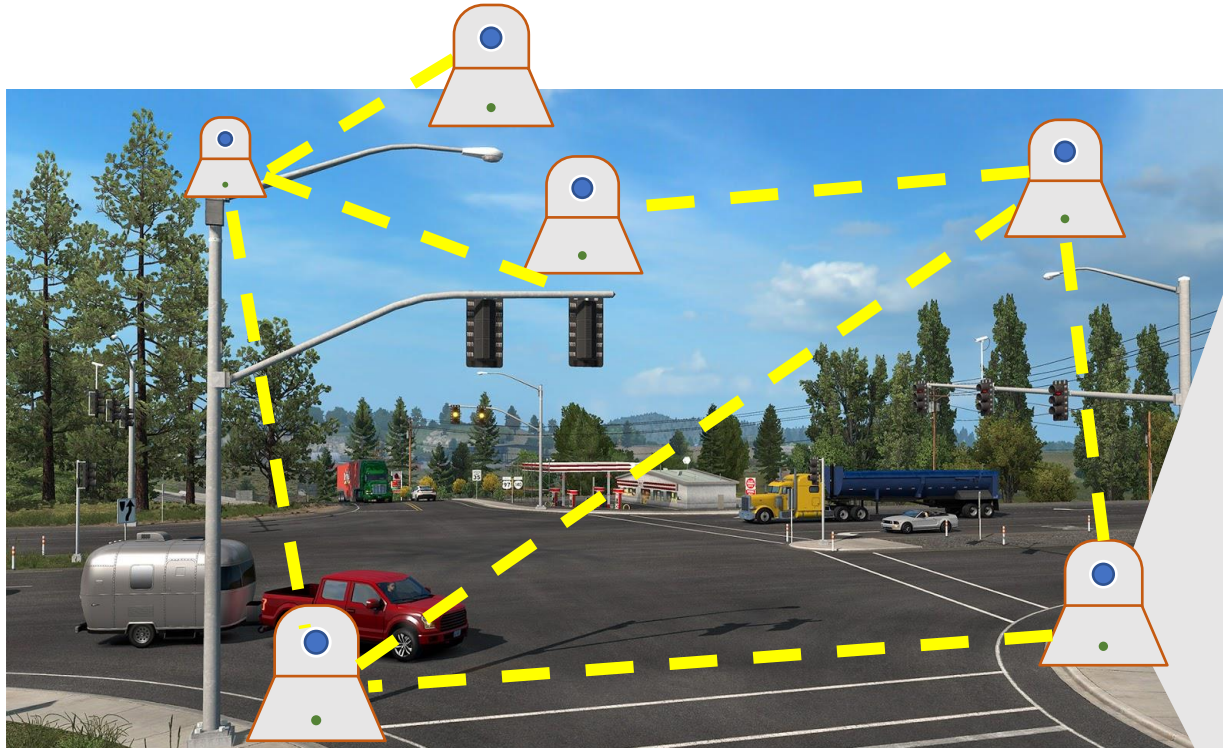


ITEM Workshop, Sep 9th 2024

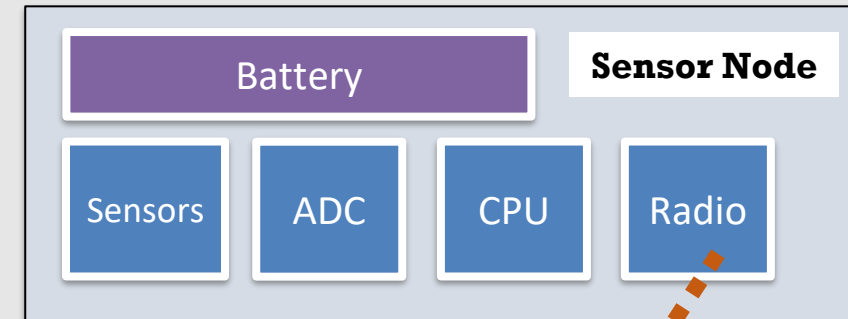
Intelligence at the Extreme-Edge



Requirements for Extreme-Edge Devices



Miniaturized **low-power** and **low-cost** sensor nodes for cyber-physical systems

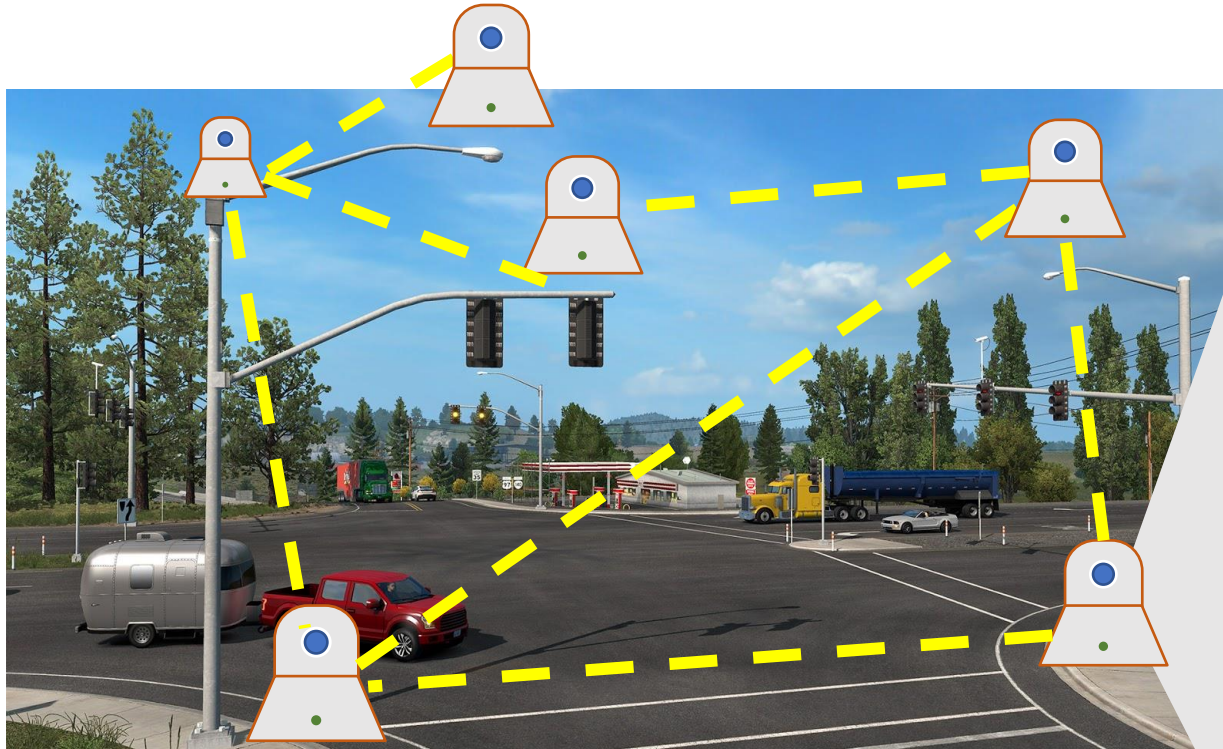


Raw Data

- ✓ simple design
- ✓ server computational power

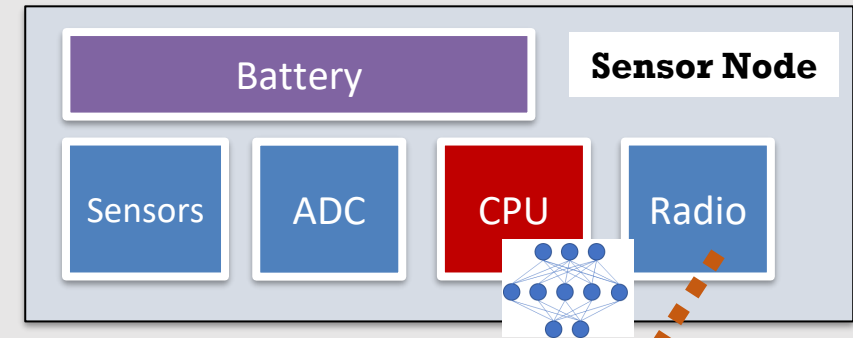
- ✗ Network scalability
- ✗ Privacy of personal data
- ✗ **Limited Energy (battery)**
 - ~40mW for 1 week from a 2000mAh battery
 - Radio Power consumption (100s mWs)

Requirements for Extreme-Edge Devices



[Credits: Unknown Author]

Miniaturized **low-power** and **low-cost** sensor nodes for cyber-physical systems



Information

Bringing Machine (Deep) Learning to Extreme-Edge Devices

Every **Smart Sensor** plays *data-to-information* conversion and transmits only low-bandwidth data to the server

✗ **Limited Energy (battery)**



Privacy by design

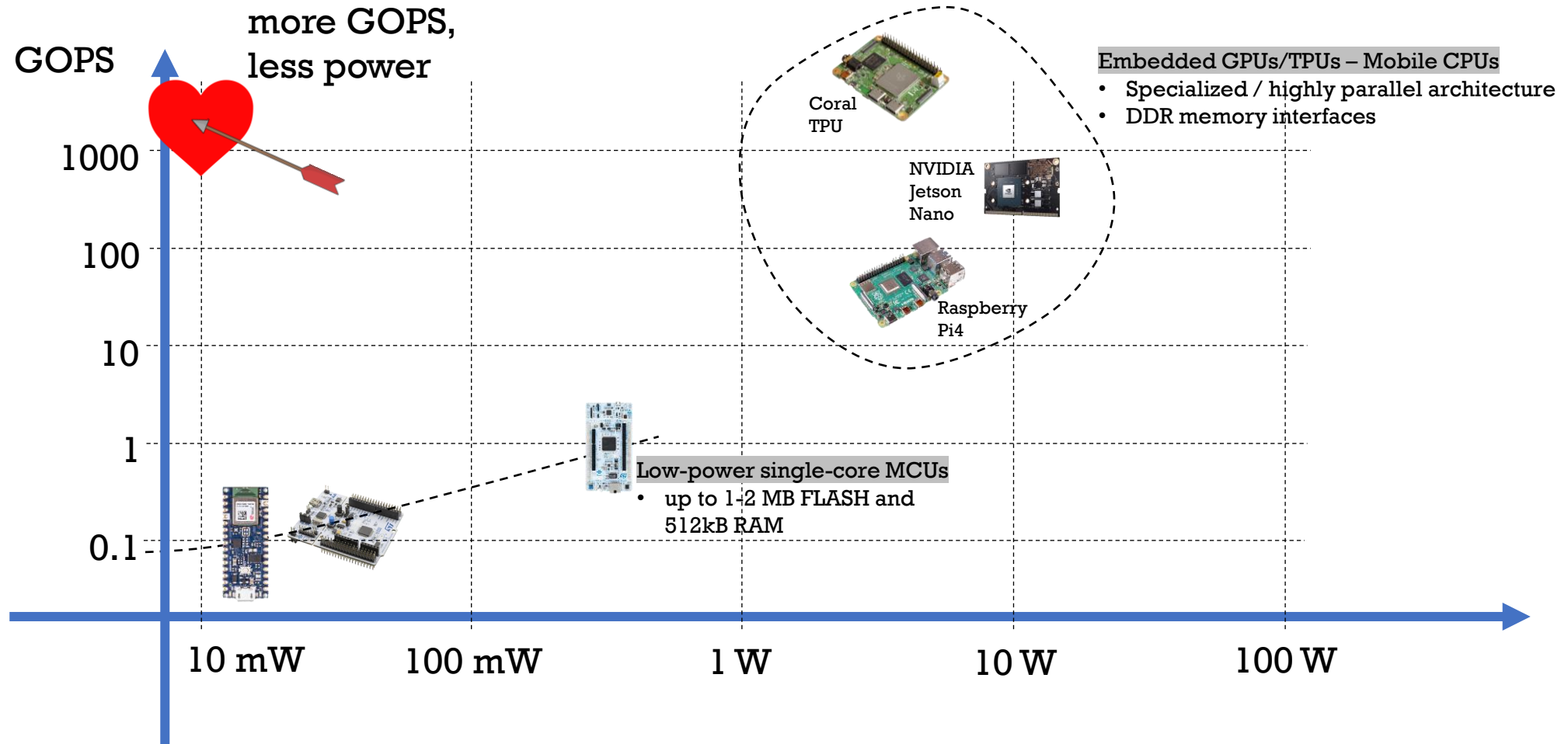


TX Bandwidth and Power Reduction



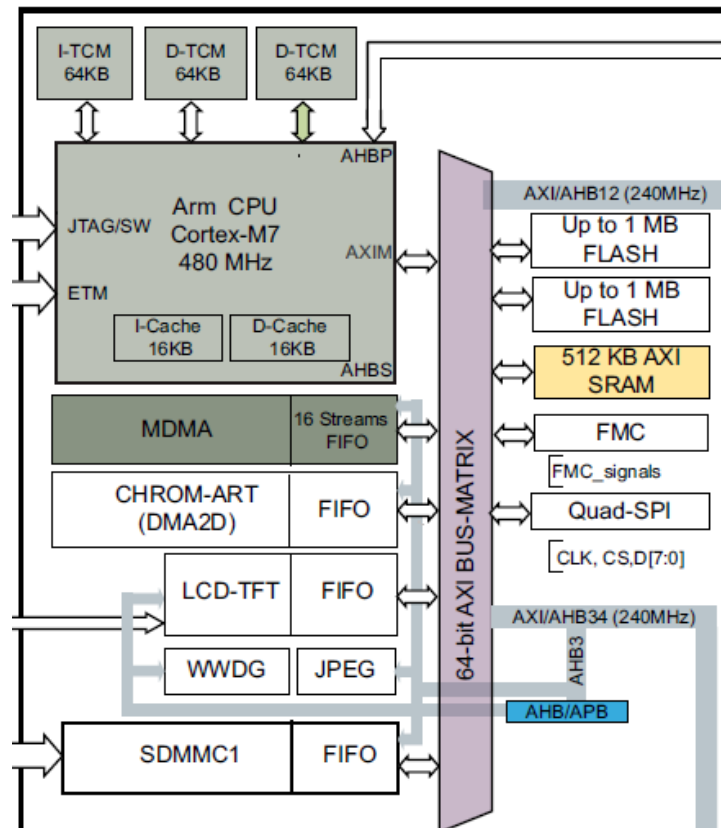
Need of Energy-Efficient TinyML devices, tools and design methods!

Processors for Extreme Edge Devices

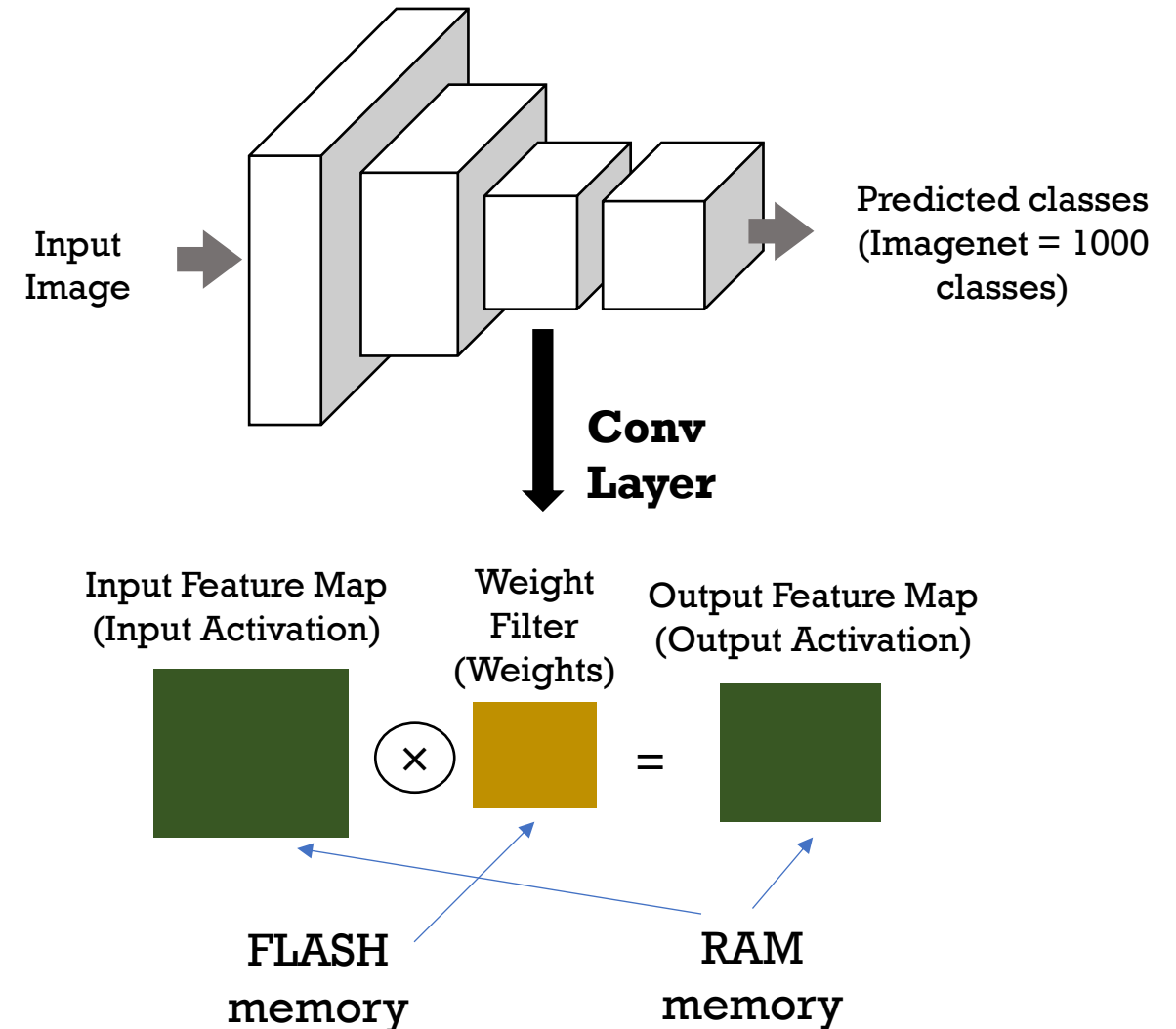


- OPS refers to peak MAC operations/s at the minimum available bitwidth. Perf from specialized accelerator not accounted.
- Estimates from available numbers may be inaccurate. Feedbacks are welcome!

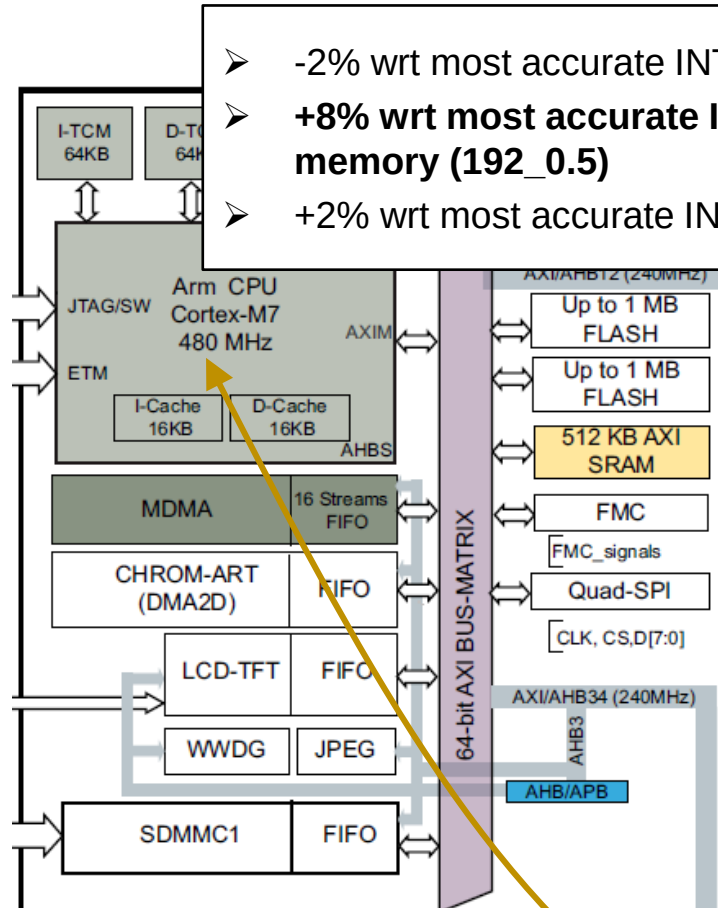
Solving Imagenet on MCUs



Source: STM32H7 datasheet



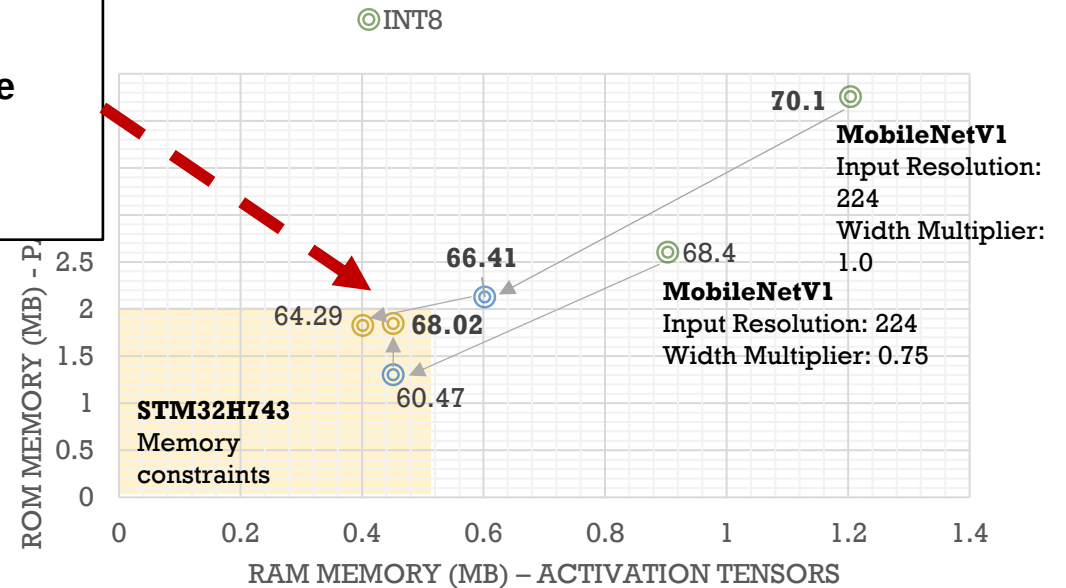
Solving Imagenet on MCUs



Source: STM32H7 datasheet

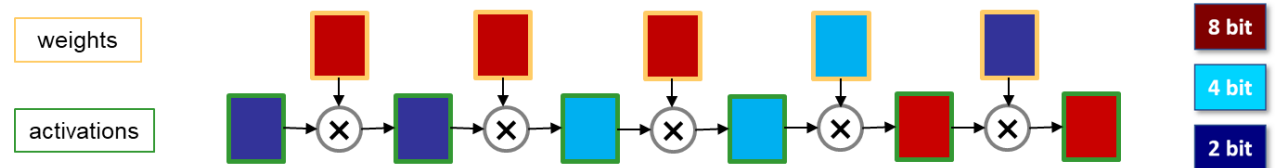
- -2% wrt most accurate INT8 mobilenet (224_1.0)
- **+8% wrt most accurate INT8 mobilenet fitting the memory (192_0.5)**
- +2% wrt most accurate INT4 mobilenet (224_1.0)

Quantization Aware Training (Top1 on Imagenet)



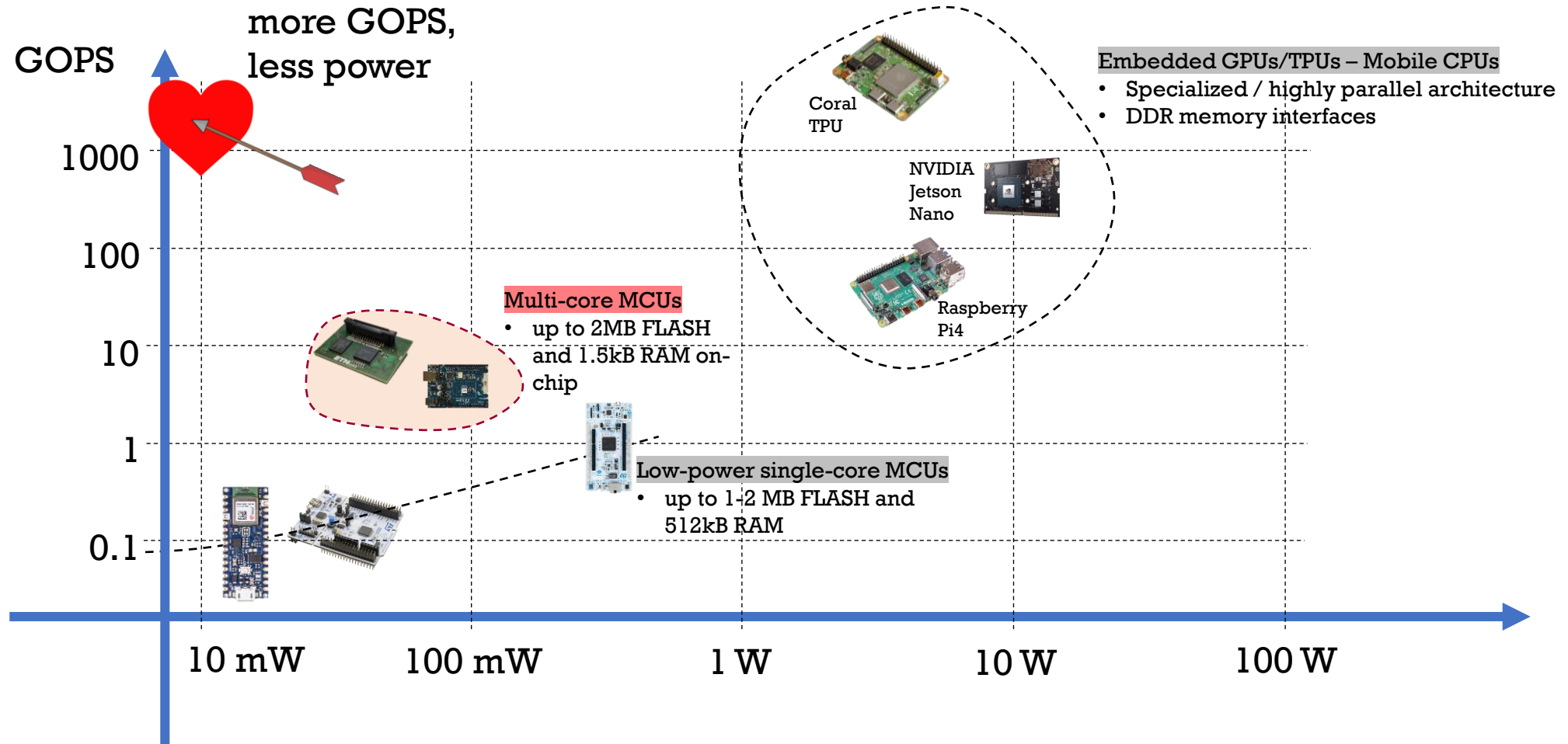
Mixed Low Precision Quantization

apply *tensor-wise quantization* to minimize accuracy degradation wrt full-precision models



but suboptimal for computation....

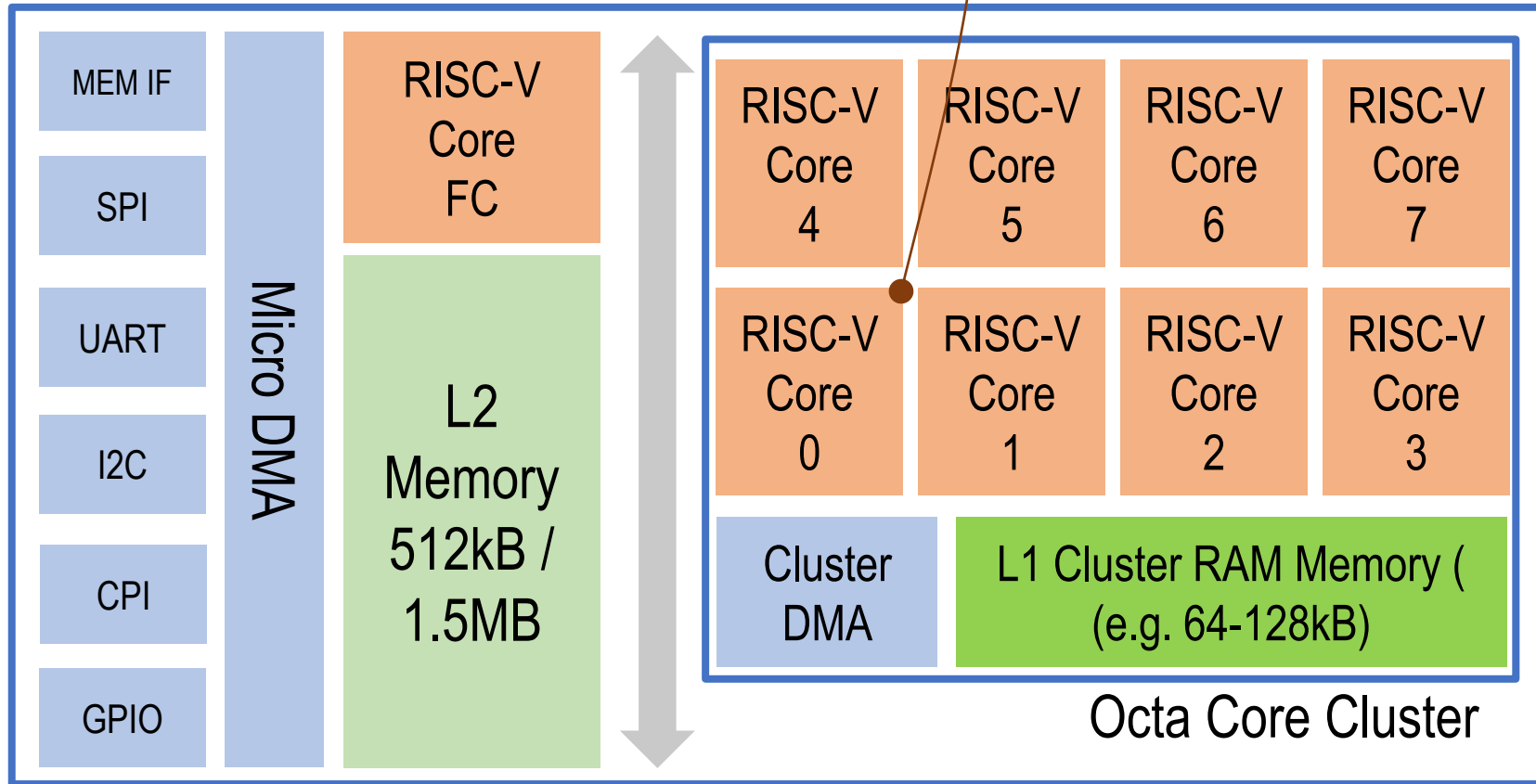
Processors for Extreme Edge Devices



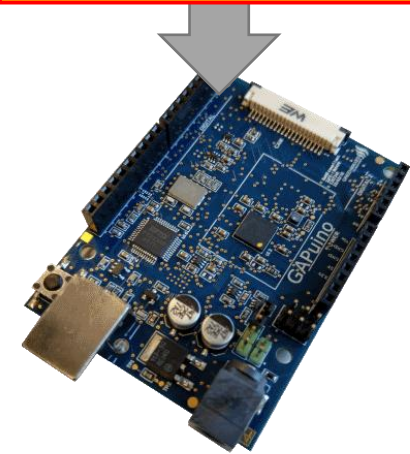
- OPS refers to peak MAC operations/s at the minimum available bitwidth. Perf from specialized accelerator not accounted.
- Estimates from available numbers may be inaccurate. Feedbacks are welcome!

An (Open) RISC-V based Multi-Core MCU

extended open RISC-V ISA to
efficiently run ML workloads



Open sourced since 2017:
github.com/pulp-platform/pulp



**Greenwaves
Technologies
GAP-8:**

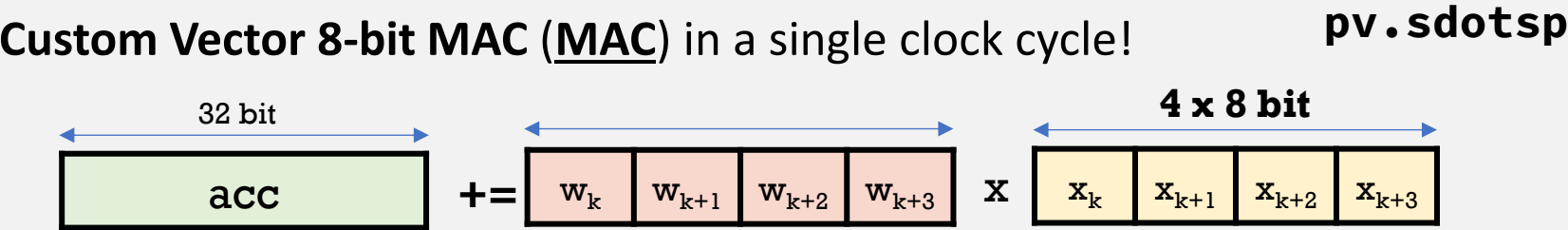
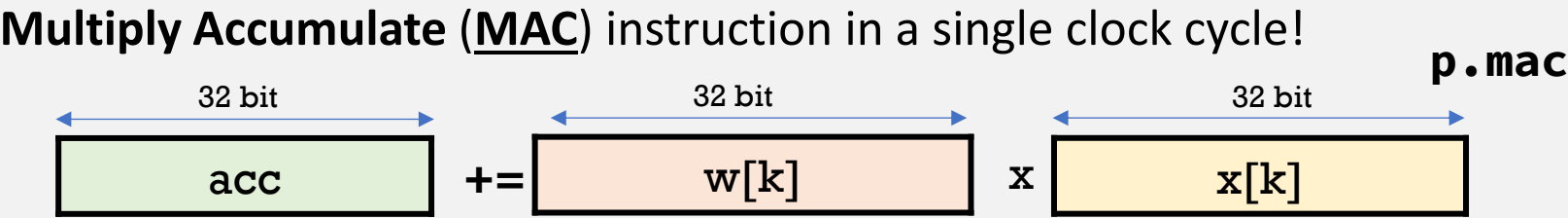
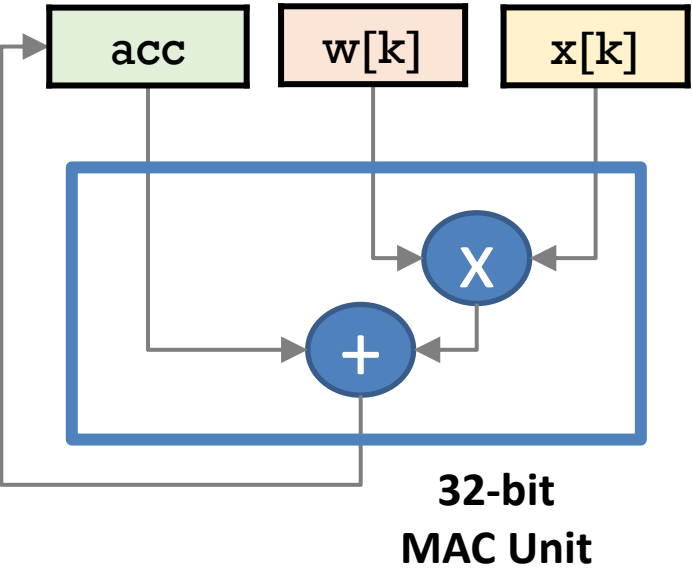
1+8 cores @
175MHz, <150mW

RISC-V benefits for Signal Processing

Algorithm	Equation
FIR Filter	$y(n) = \sum_{k=0}^M a_k \cdot x(n - k)$
Convolution	$y(n) = \sum_{k=0}^M x(k) \cdot h(n - k)$
DFT	$X(k) = \sum_{n=0}^{N-1} x(n) e^{-j(\frac{2\pi}{N})nk}$
DCT	$F(u) = \sum_{x=0}^{N-1} c(u) f(x) \cos[\frac{\pi}{2N} u(2x + 1)]$

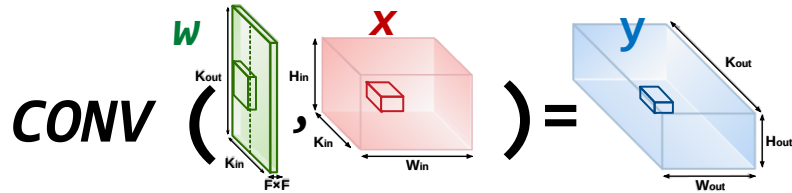
Signal-processing workloads are dominated by linear algebra kernels and **dot-products !!**

$$\text{acc} = \sum_{k=0}^N w_k \cdot x_k$$



4x speed-up

PULP-NN: Accelerating DNN Inference

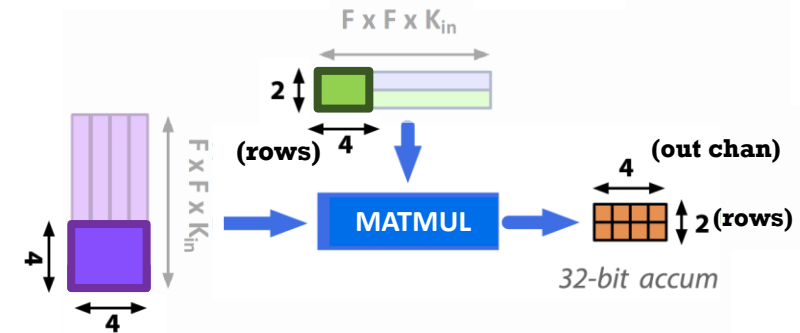


```

Parallel loop
parallel for i,j in range(0, H_out, W_out):
    for m in range(0, K_out):
        for ui in range(0, F):
            for uj in range(0, F):
                for n in range(0, K_in):
                    im2col[ui*F*K_in + uj*K_in + n] =
                        x[i+ui, j+uj, n]
                psum = 0
                for idx in range(0, F*F*K_in):
                    psum += w[m, idx] * im2col[idx]
                y[i, j, m] = act(psum)
im2col
GEMM
Generic
Mat Mul
    
```

- ❑ Peak Performance of **15.5 MAC /clk** on **8 cores**
- ❑ **19.6x** and **30x** faster than CM7 and CM4 based MCUs using CMSIS-NN for CIFAR10 inference using 16-bit MAC

Optimized GEMM on PULP by 1) **parallelism** 2) exploiting **SIMD MAC instructions** and 3) maximizing **data reuse in register file**



```

lp.setup
p.lw w0, 4(W0!) // hwloop init
p.lw w1, 4(W1!) // load 8b vector
p.lw w2, 4(W2!)
p.lw w3, 4(W3!)
p.lw x1, 4(X0!)
p.lw x2, 4(X1!)
pv.sdotsp acc1, w0, x0
pv.sdotsp acc2, w0, x1
pv.sdotsp acc3, w1, x0
pv.sdotsp acc4, w1, x1
pv.sdotsp acc5, w2, x0
pv.sdotsp acc6, w2, x1
pv.sdotsp acc7, w3, x0
pv.sdotsp acc8, w3, x1
end
    
```

for each core

Load 16 weights (8-bit)
4 out chan, 4 in chan
address post-increment

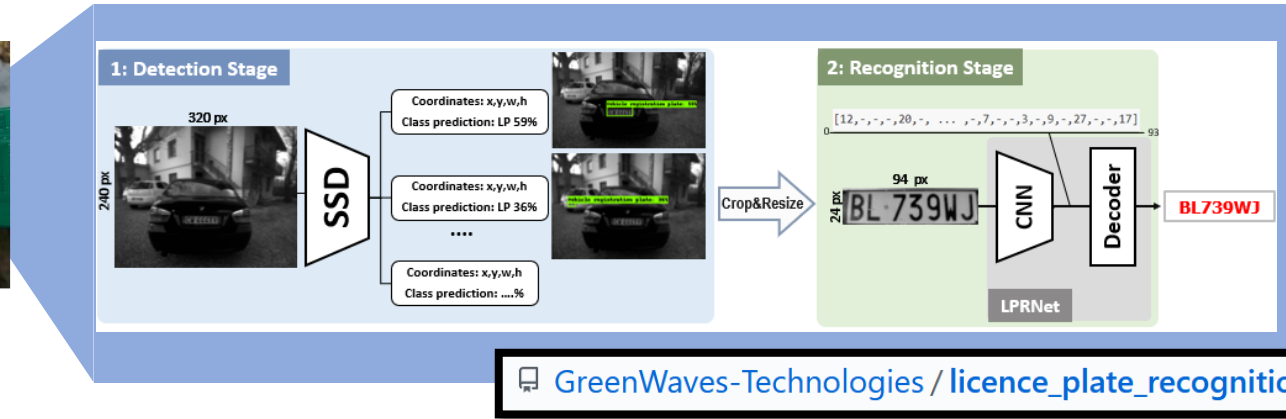
Load 8 pixels
2 rows, 4 in chan
address post-increment

Compute 32 MAC over 8 accumulators using **vectorized dot-product instructions**

Enabling Vision Object Detection on MCUs



L. Lamberti et al., ISCAS 2021



- 1.1 FPS inference @ 175MHz, performing **687M MAC**.
 - highest complexity on MCUs**
- 117mW power envelope, 108 mJ per inference.
 - 73x less energy w.r.t. previous systems**



L. Lamberti et al., DATE 2023

[pulp-platform](#) / [pulp-detector](#)

Random Exploration	Detection Acc @ vel [m/sec]			FPS
	0.1	0.5	1.0	
SSD 1.0x	27%	90%	83%	1.6
SSD 0.75x	27%	80%	37%	2.3

Object Detection: **SSDLite Mobilenet on GAP8**:
134mW / 7W (tot nano-drone budget)

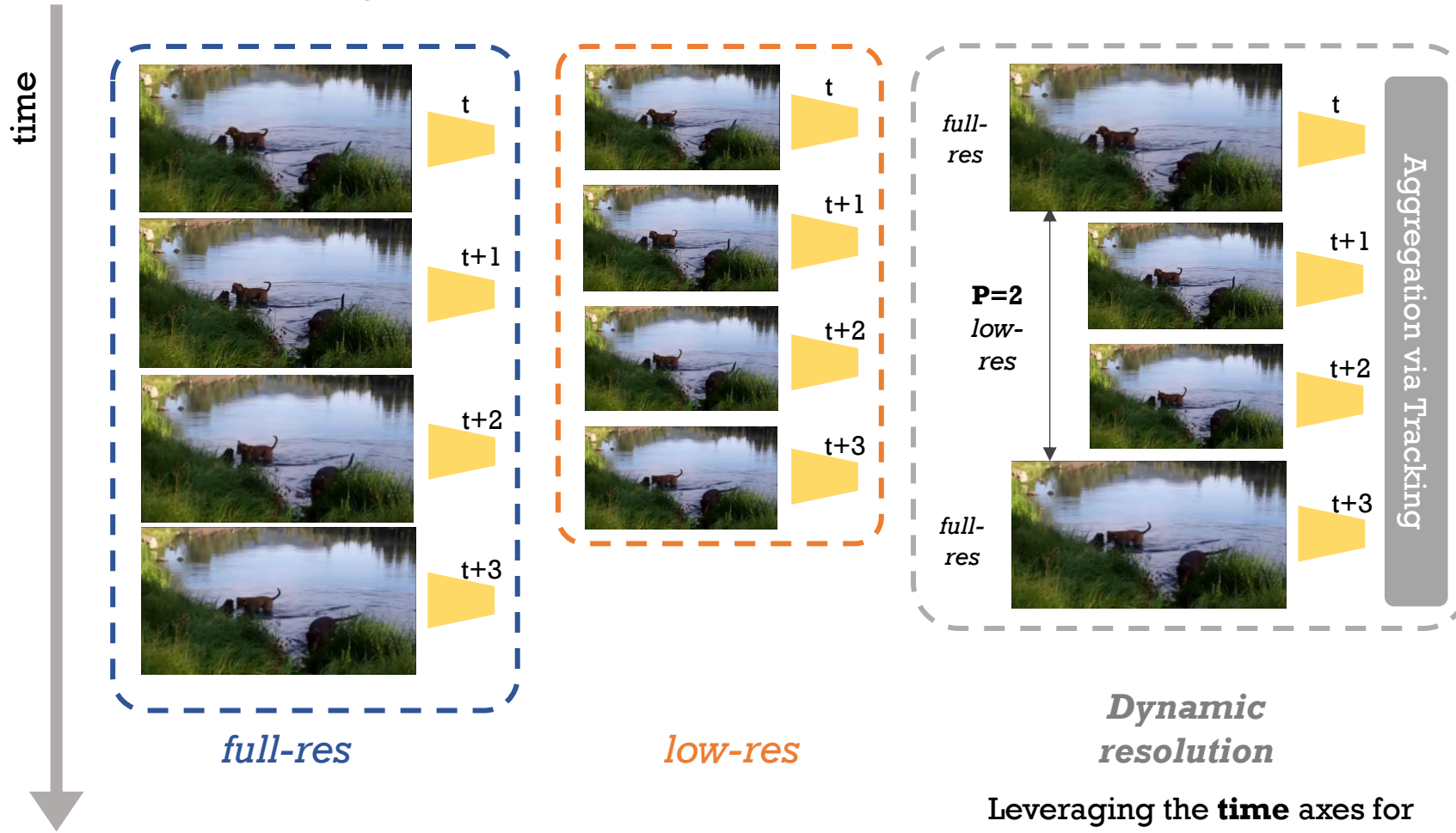
Bio-inspired Autonomous Exploration Policies with CNN-based Object Detection on Nano-drones

Lorenzo Lamberti, Luca Bompani, Victor Javier Kartsch, Manuele Rusci, Daniele Palossi, Luca Benini

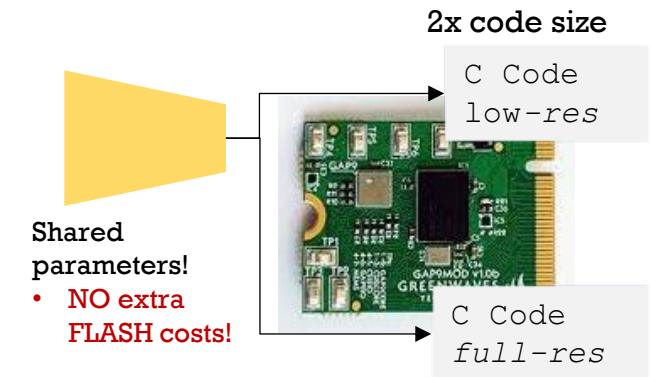
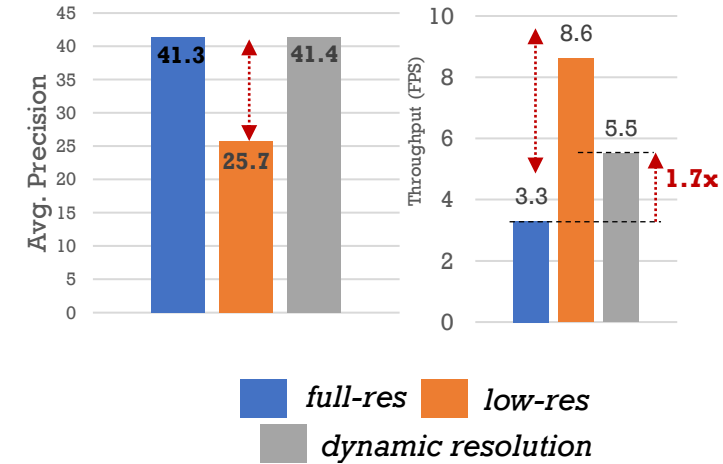


Dynamic Multi-Resolution

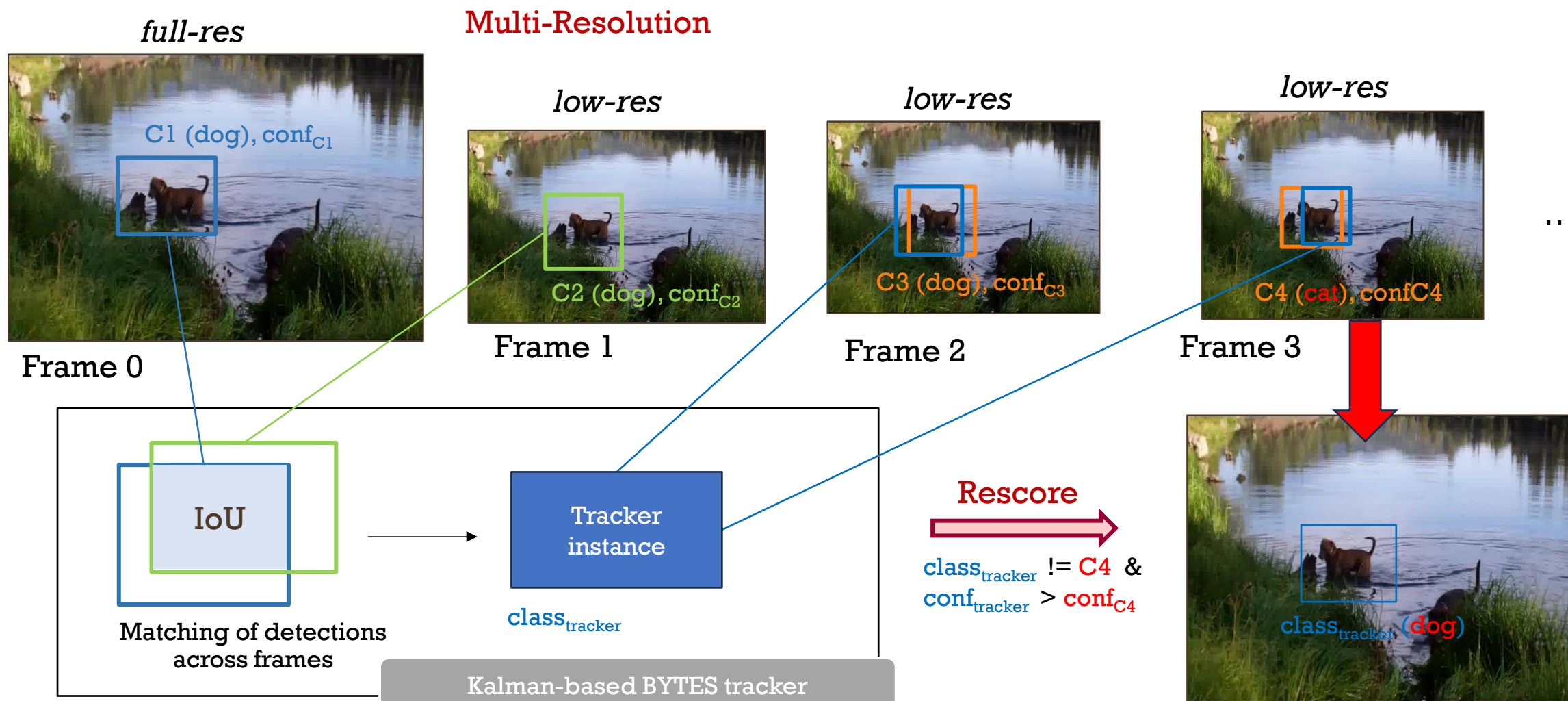
Imagenet-Vid sequences



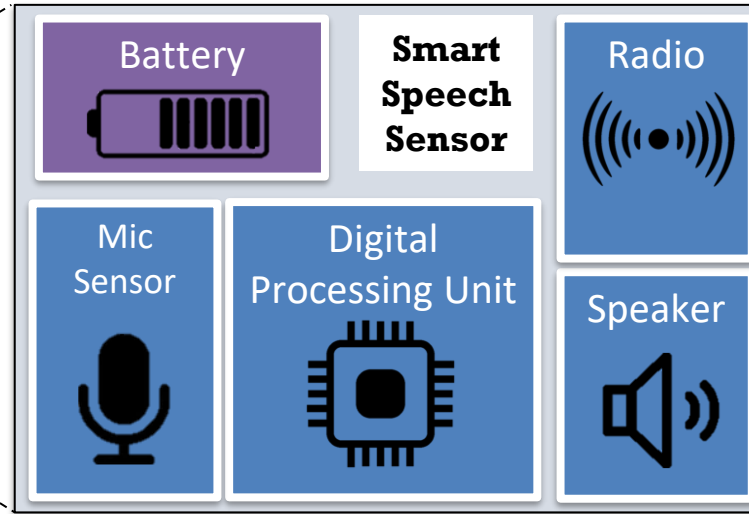
YOLO-X nano



Multi-Resolution Rescored ByteTrack

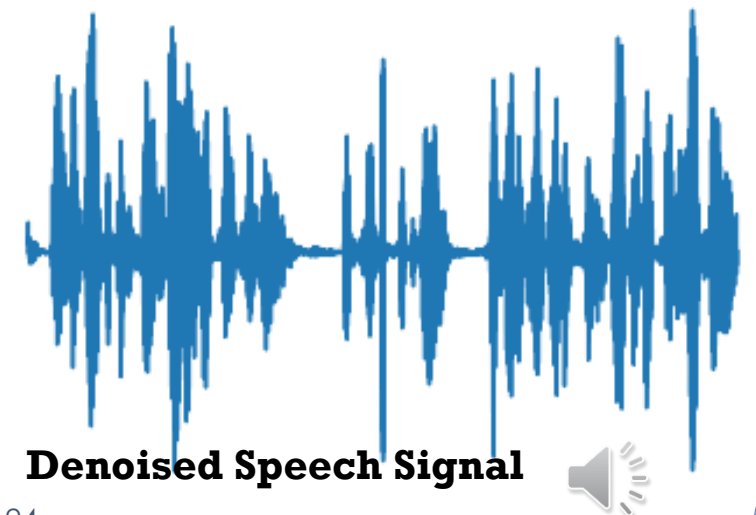


Speech Enhancement for Hearables

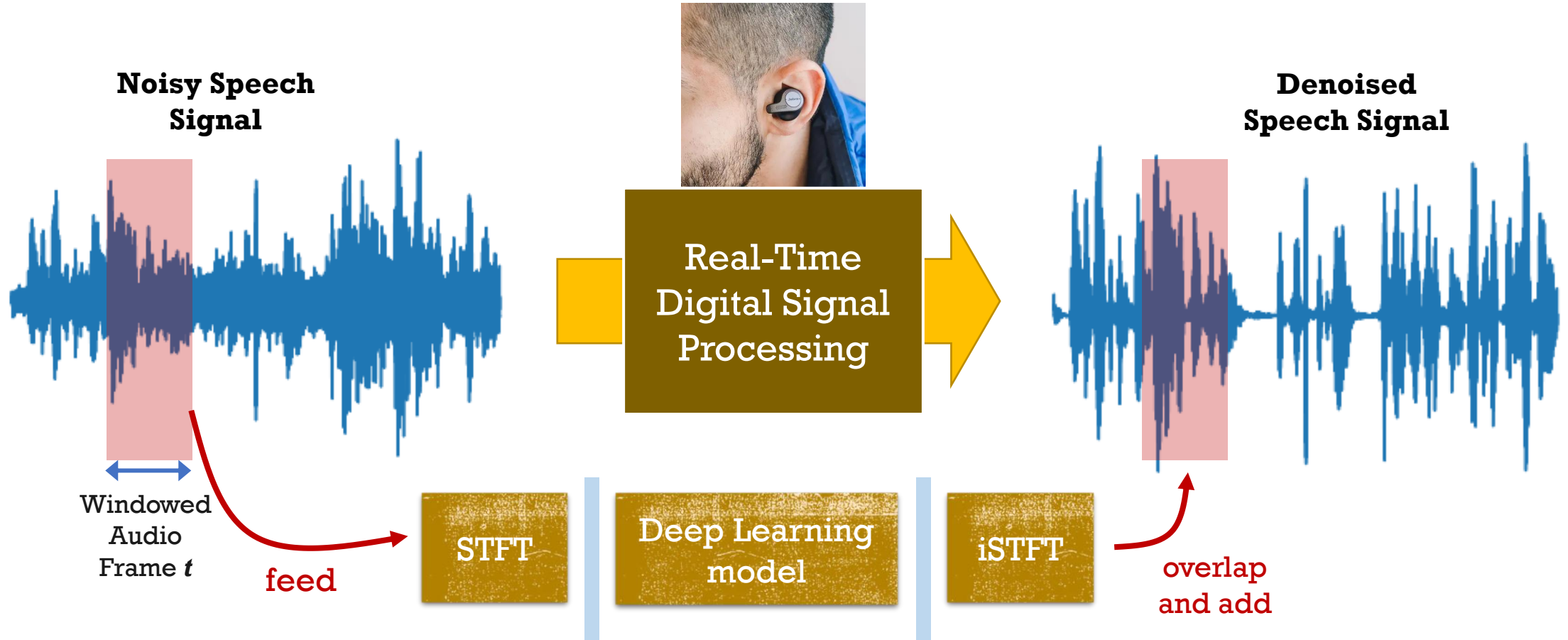


Low-Power Crucial

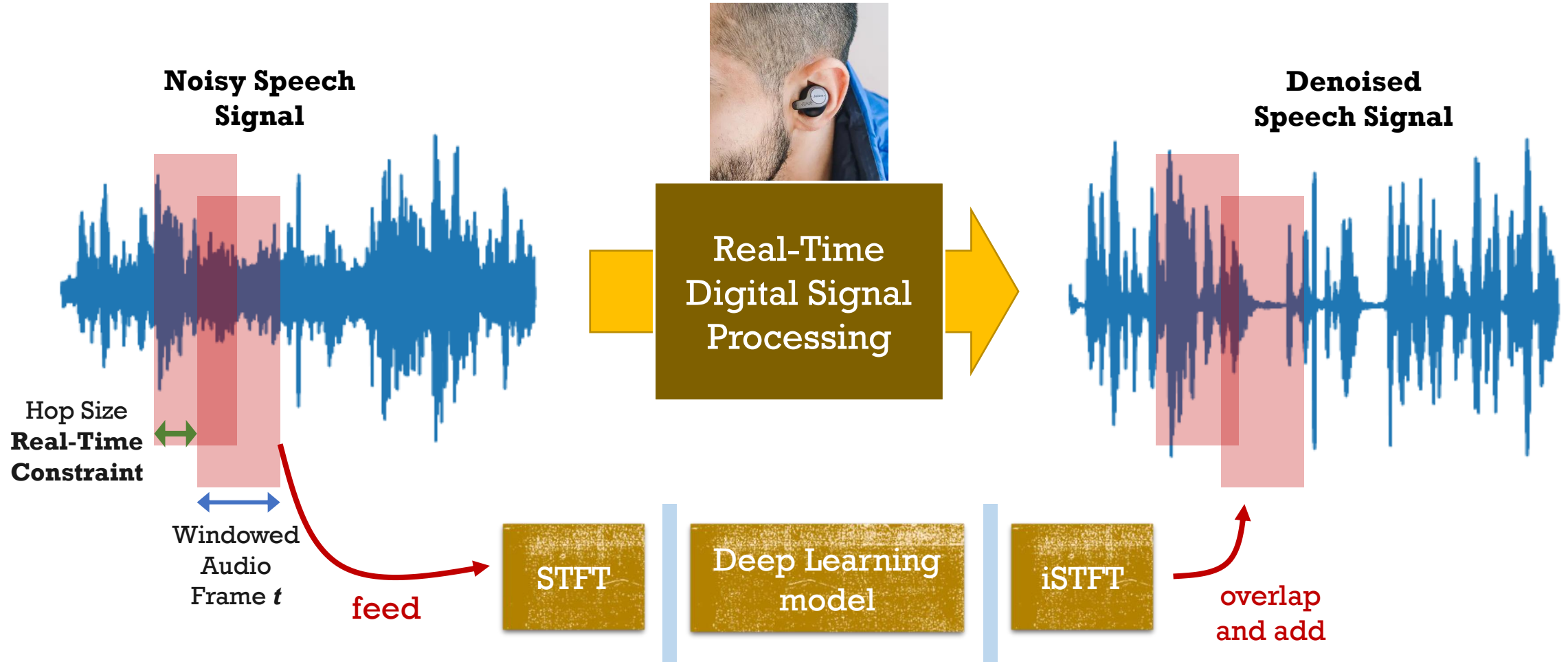
- size constraints demand small batteries (e.g. 60mAh)
- **20h** if avg power of 10mW for sensing, processing and actuation



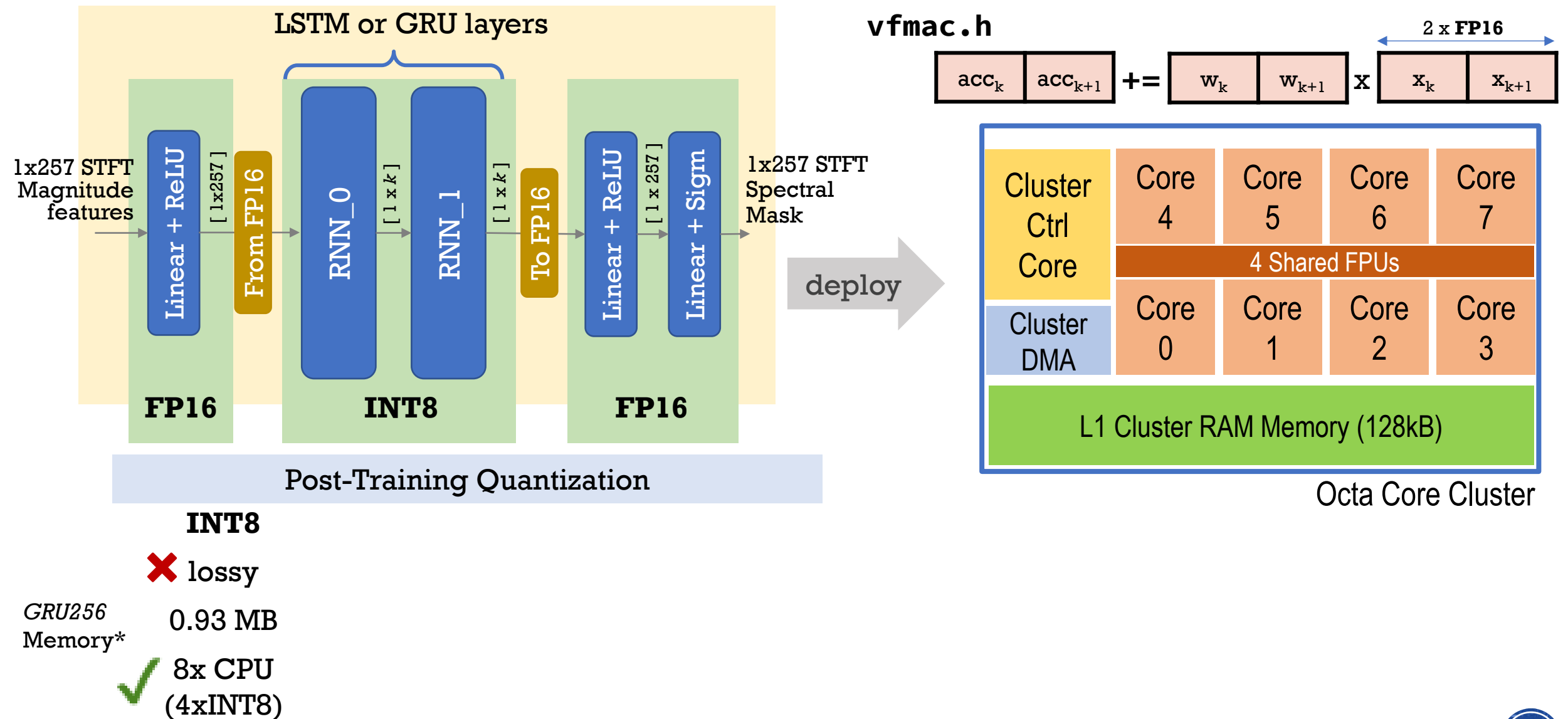
Speech Enhancement for Hearables



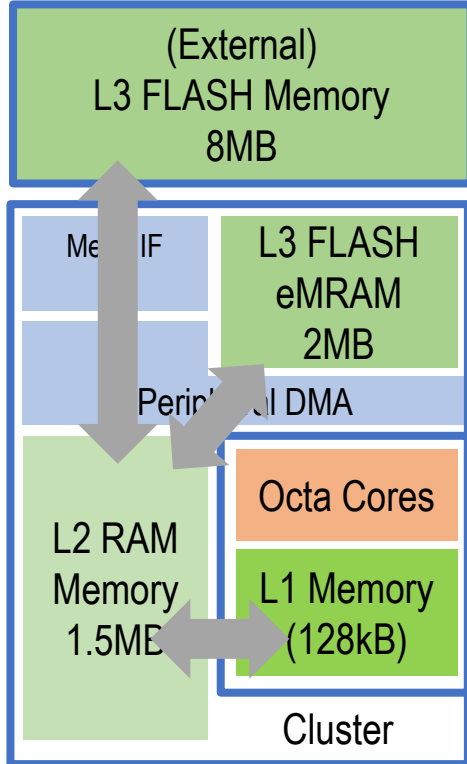
Speech Enhancement for Hearables



RNN-based SE on Multi-Core MCUs

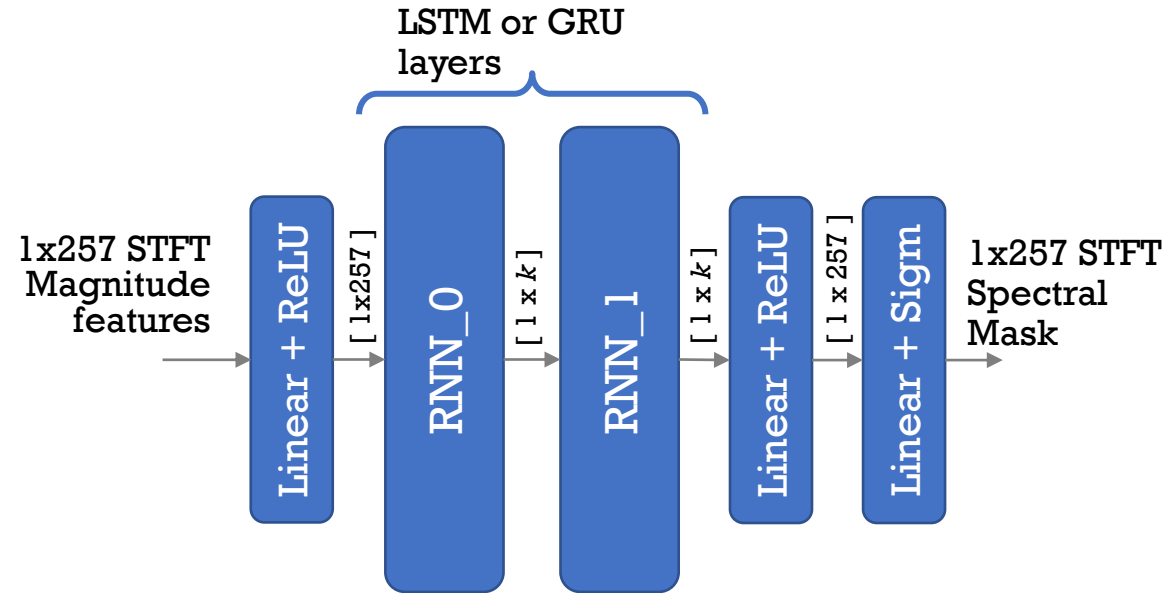


Latency & Power on target HW/SW



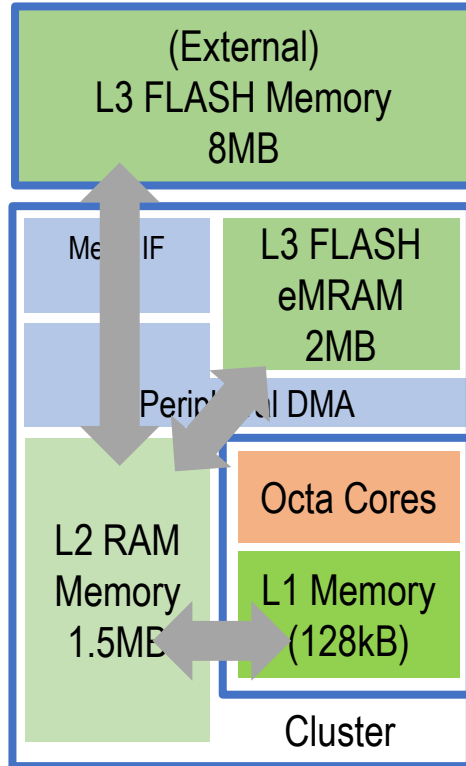
Parameters are initially stored to large L3 memories but can be *promoted* to L2 on-chip memory

$$\rho^{L3} = \frac{\# L3 \text{ params}}{\# \text{ params}}$$

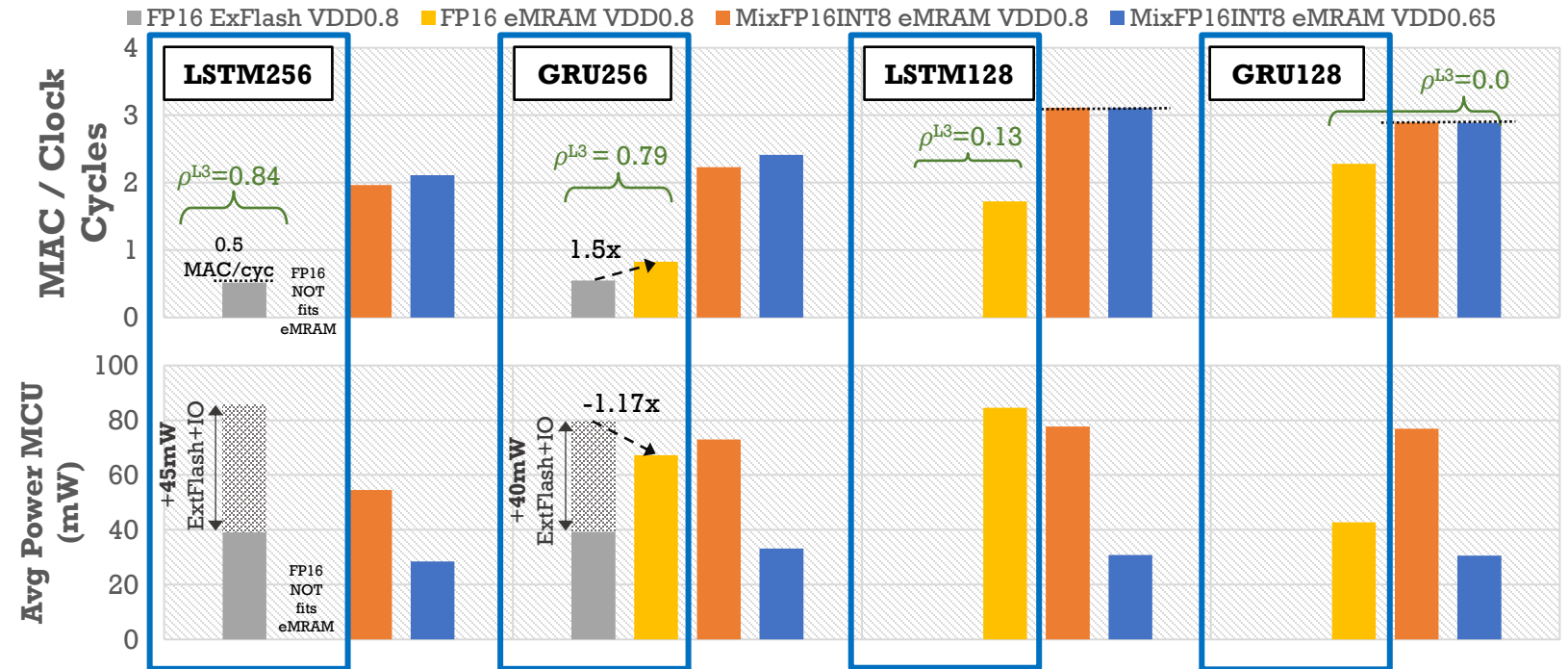


	LSTM256	GRU256	LSTM128	GRU128
k	256	256	128	128
RNN_0	LSTM(257,256)	GRU(257, 256)	LSTM(257,128)	GRU(257, 128)
RNN_1	LSTM(257,256)	GRU(257, 256)	LSTM(128,128)	GRU(128, 128)
Params	1.24 M	0.985 M	0.493 M	0.411 M

Latency & Power on target HW/SW



eMRAM as internal Flash Memory

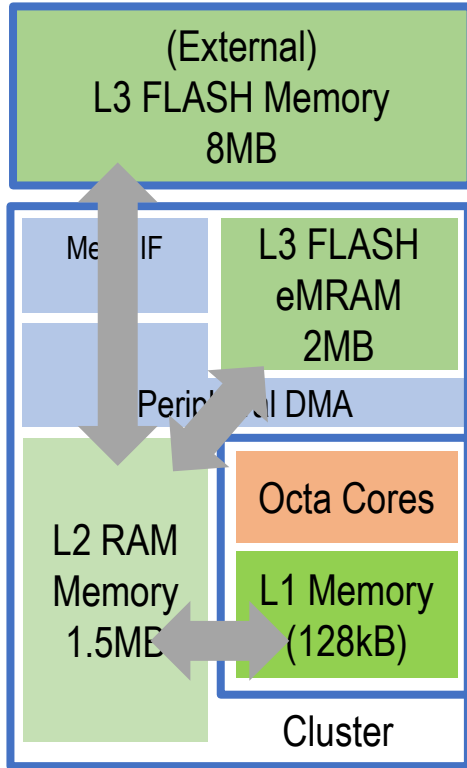


Parameters are initially stored to large L3 memories but can be *promoted* to L2 on-chip memory

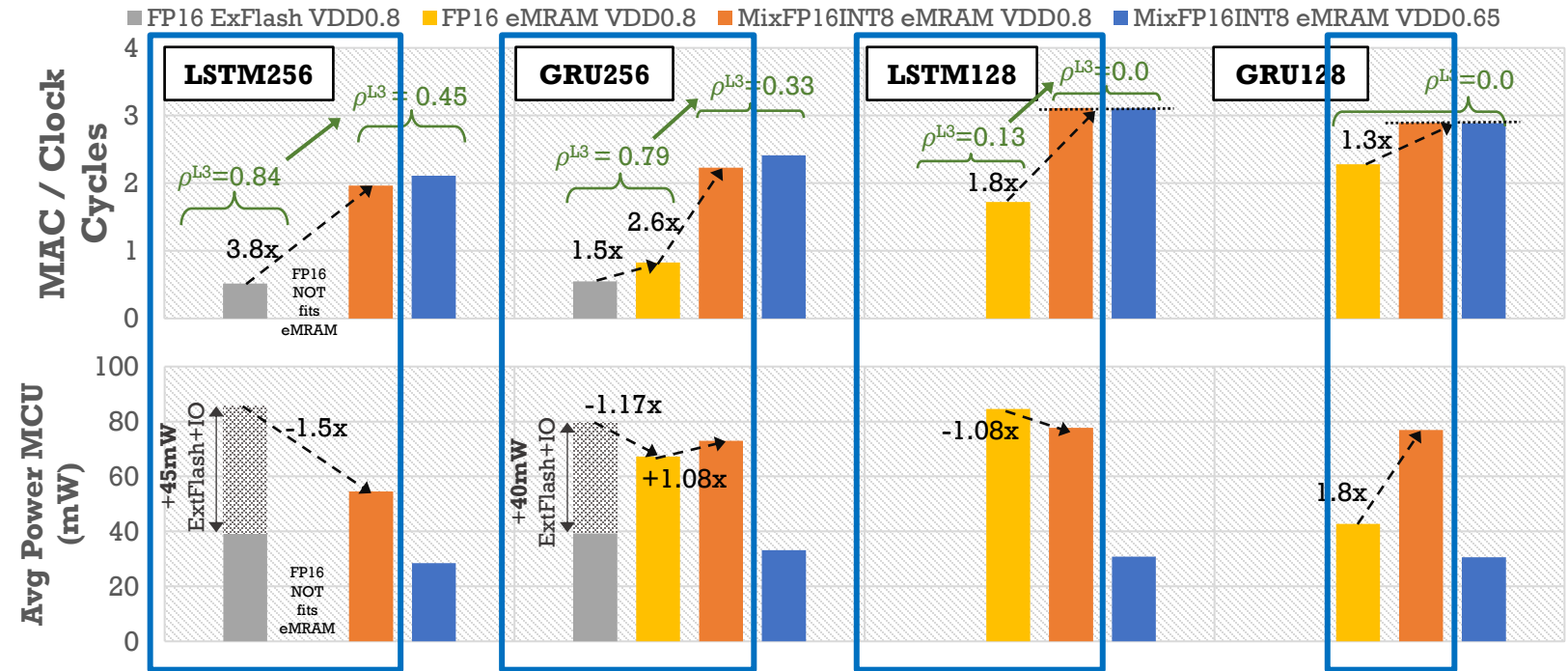
$$\rho^{L3} = \frac{\# L3 \text{ params}}{\# \text{ params}}$$

- FP16 LSTM256 and GRU256 are **L3 memory bound** (+45 mW of ext L3)
- FP16 GRU256 cut power memory costs by storing data in 2MB on-chip FLASH
- Thanks to promotion, FP16 LSTM128 and GRU128 have low ρ^{L3}
 - efficiency up to 2.2 MAC/cyc

Latency & Power on target HW/SW



eMRAM as internal Flash Memory



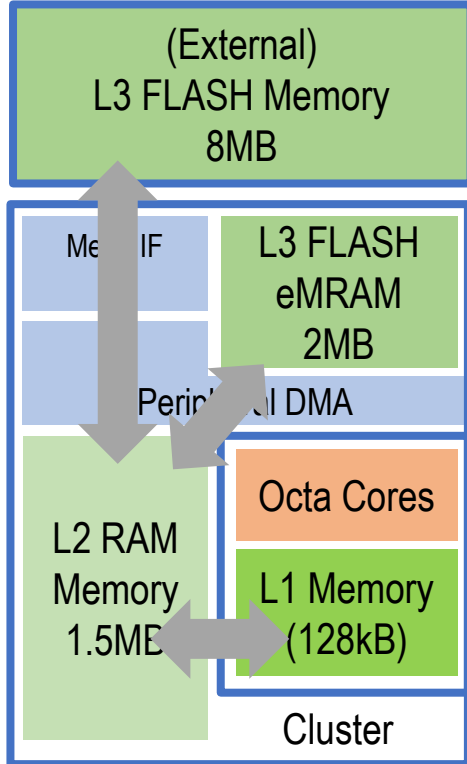
Parameters are initially stored to large L3 memories but can be *promoted* to L2 on-chip memory

$$\rho^{L3} = \frac{\# L3 \text{ params}}{\# \text{ params}}$$

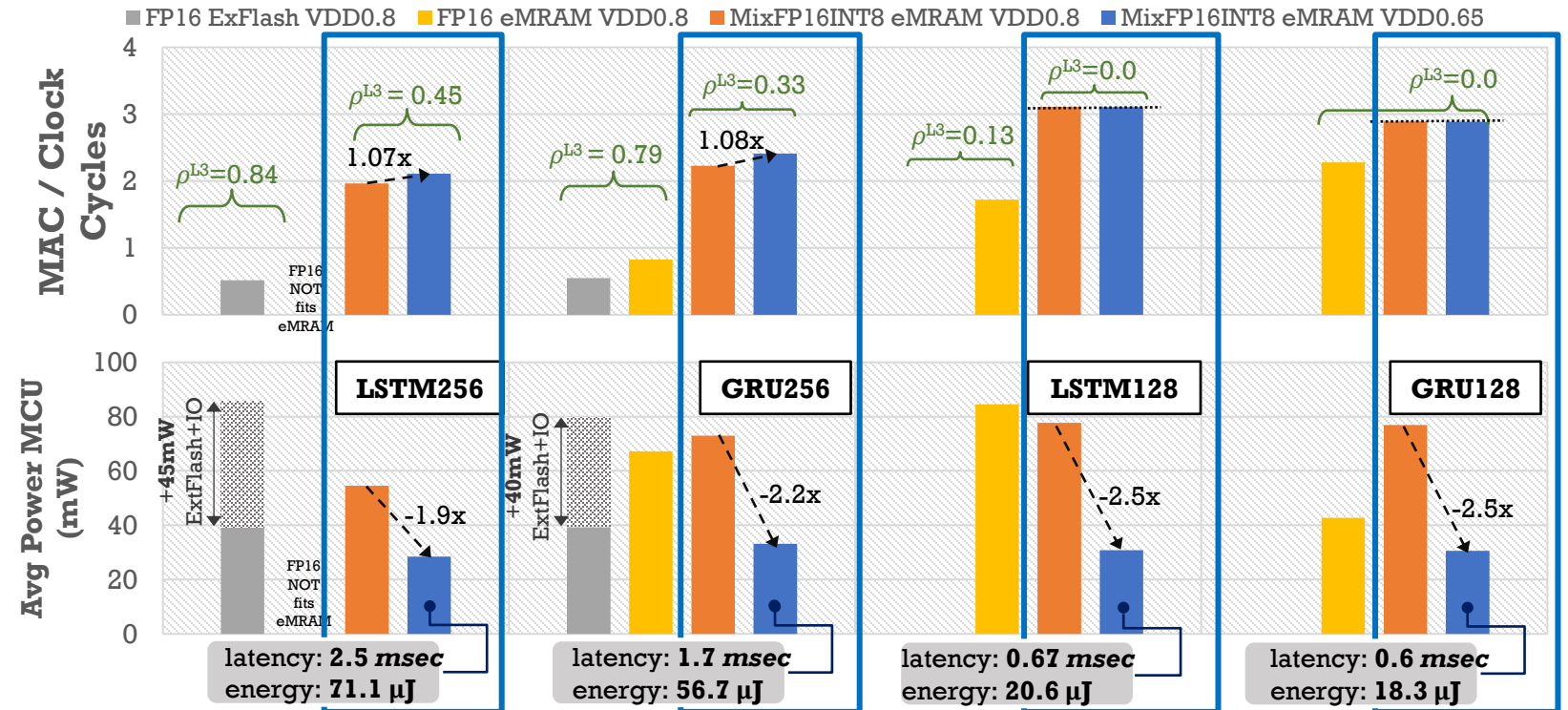
- LSTM256 and GRU256 decrease ρ^{L3} thanks to Mixed-Precision
 - avg. eff. up to 2.2MAC/cyc
 - higher power cost because of higher operation density

- LSTM128 and GRU128 improves MAC/cyc thanks to Mixed Precision

Latency & Power on target HW/SW



eMRAM as internal Flash Memory

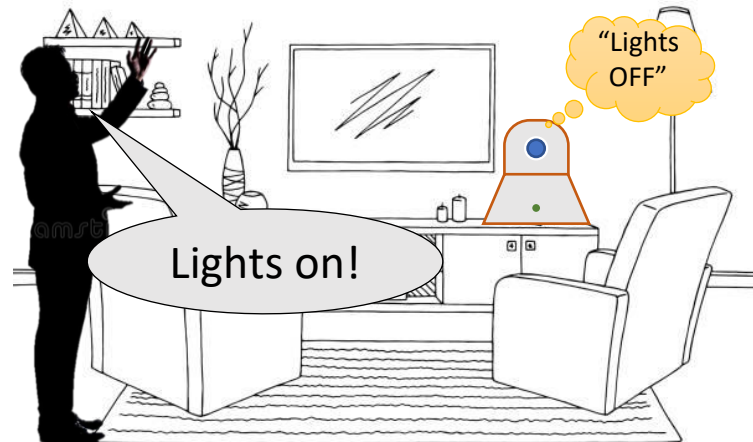
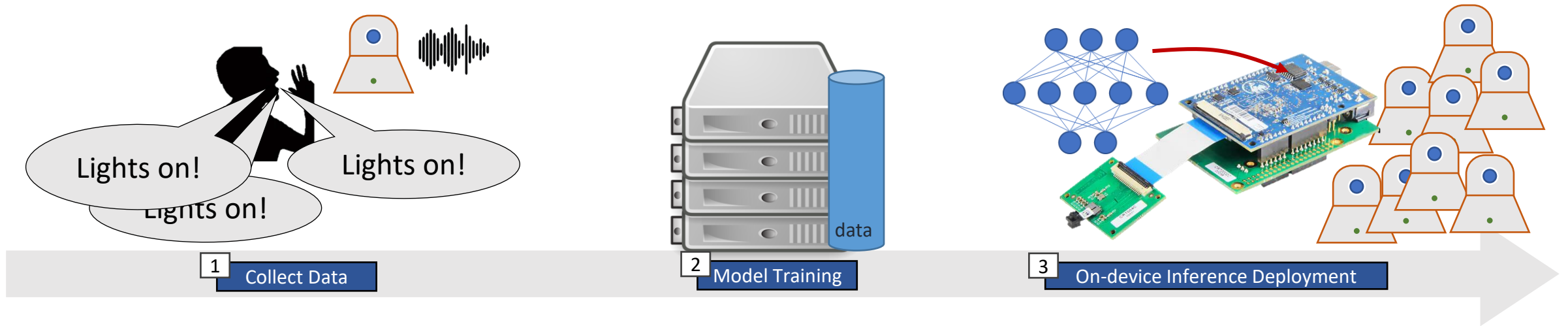


Parameters are initially stored to large L3 memories but can be *promoted* to L2 on-chip memory

$$\rho^{L3} = \frac{\# L3 \text{ params}}{\# \text{ params}}$$

- ❑ Scaling down the voltage reduces the power by 2-2.5x
- ❑ Real-time constraint (T=6.25msec) matched
- ❑ >10x more energy efficient than previous solutions

Train-Once-Deploy-Everywhere



Real-world
experience

.... is not enough!

- Generalization to unknown environments
 - original data $\neq$ target data
 - Model capacity
- Domain shifts over time
- Sensor-specific data
- **Personalization**
 - E.g. new objects, new keywords (new classes)



Training Data
(Google Open Images)



Real Data
(Low-Power Camera)

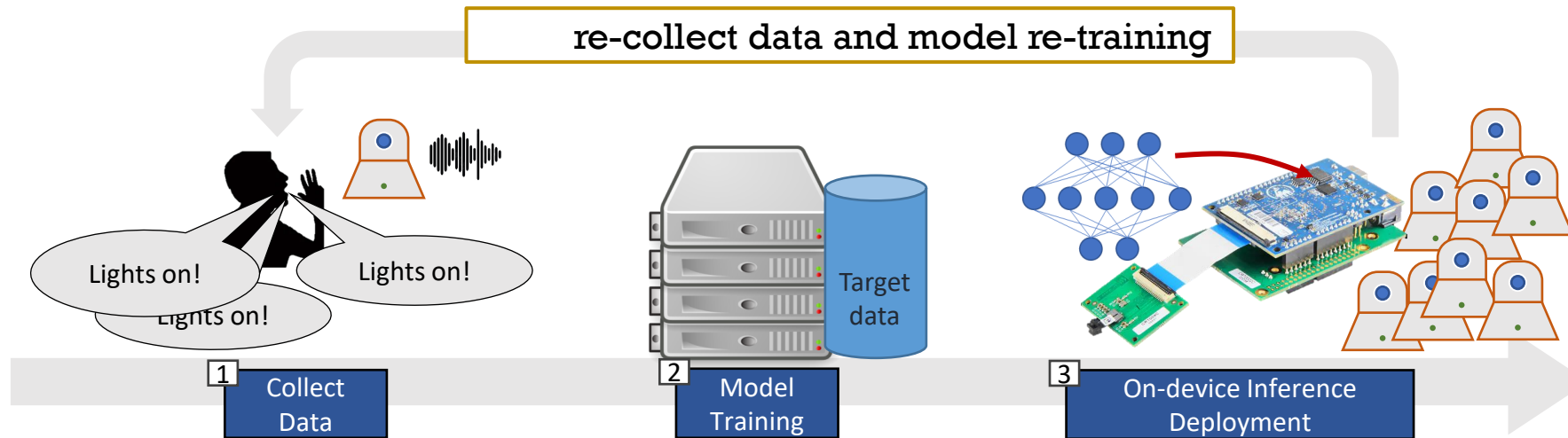


Obj Detector trained on
Google Open Images

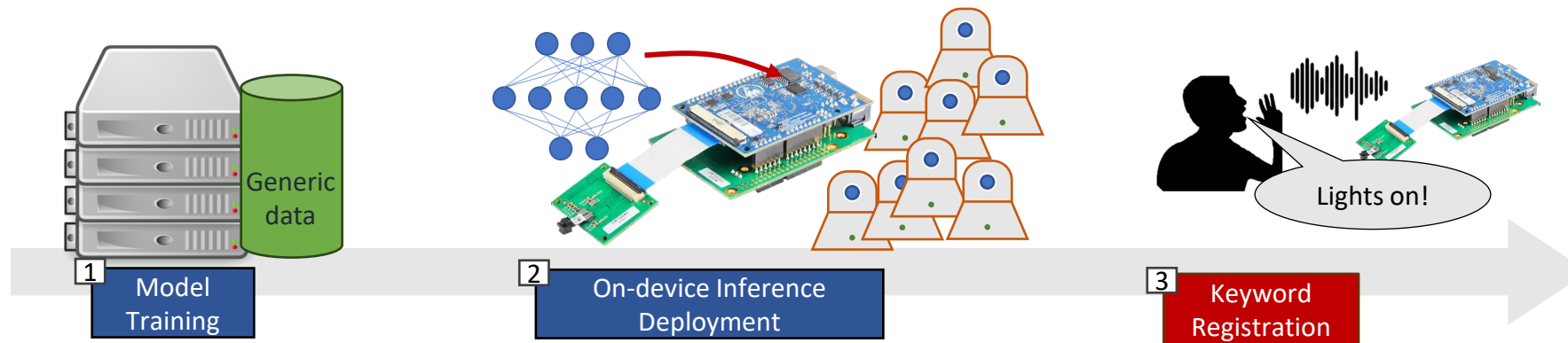


After finetuning on
Low-Power Camera data

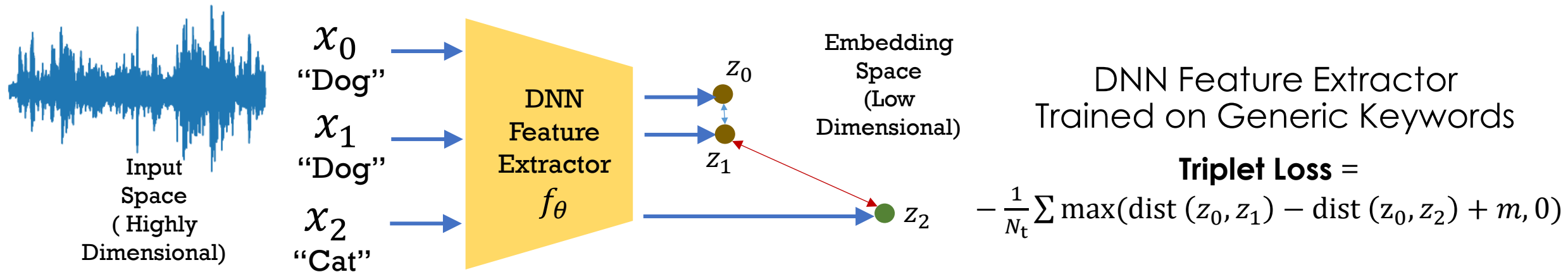
Keyword Spotting Personalization



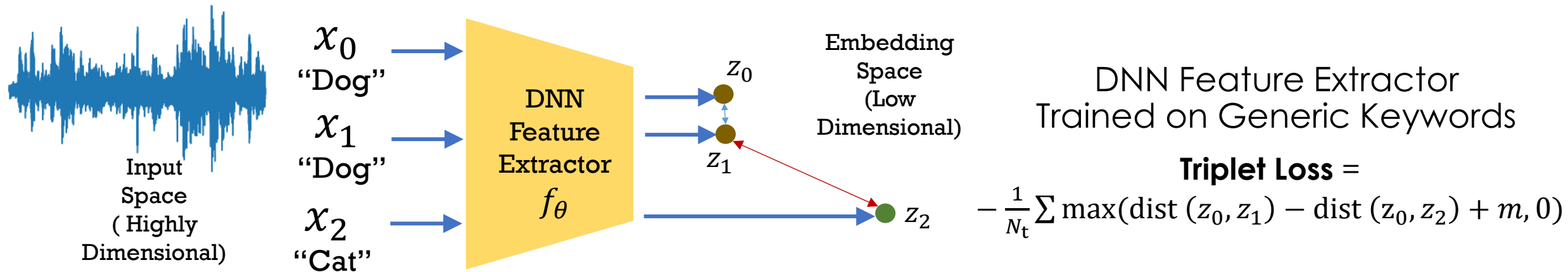
Customization on-device!



Few-Shot Learning



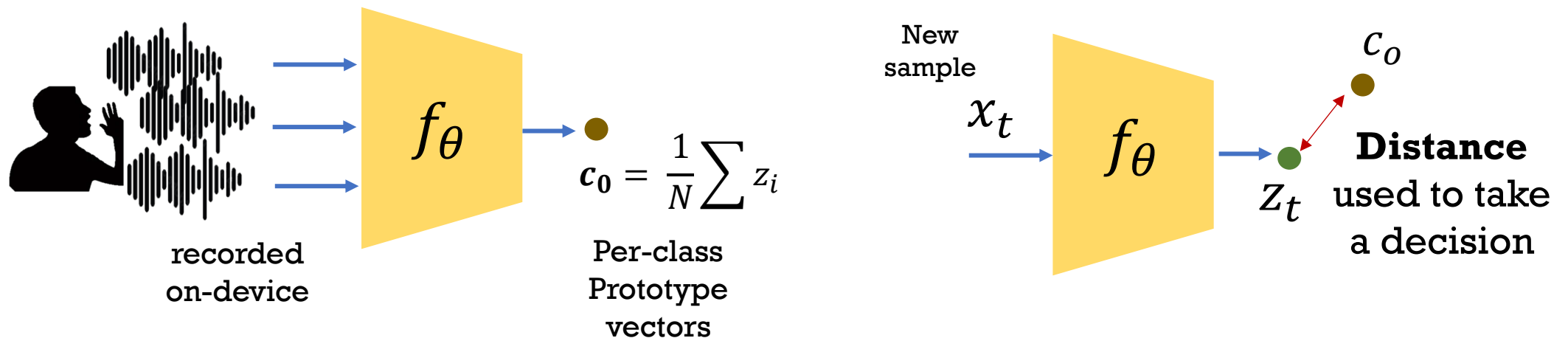
Few-Shot Learning



Few Shot Learning (FSL) On-Device

- [1] Initialize your classifier
- N shots (multiple keywords)
 - reconfigured at runtime (no server)

[2] Runtime Decision



Experimental Results

Models trained on MSWC
and tested on GSC

Open-Set Setting

$$y = \begin{cases} \arg \max p_i, & p_i > \gamma \\ y_o \text{ (unknown class)} \end{cases}$$

Accuracy at False Acceptance Rate = 5%

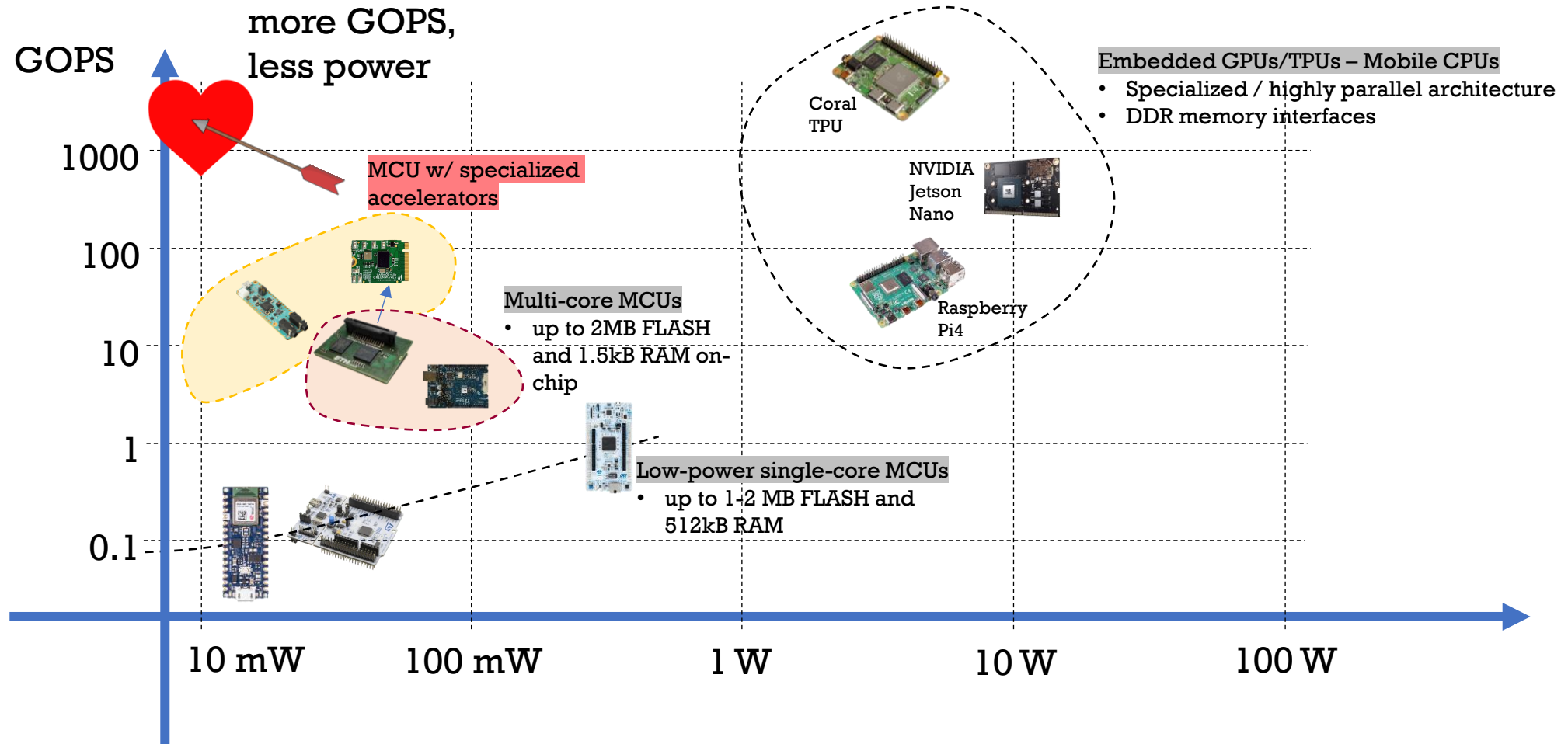
Models	Mem Params	Max RAM	MMAC	1 shot	3 shots	5 shots	10 shots
TCRES8	61 k	23 k	3.00	0.38	0.56	0.63	0.67
DSCNN S	21 k	24 k	2.66	0.37	0.57	0.65	0.69
DSCNN M	132 k	52 k	9.64	0.45	0.64	0.71	0.74
DSCNN L	407 k	112 k	28.05	0.47	0.70	0.75	0.77
RES15	482 k	537 k	235.11	0.55	0.74	0.78	0.80

Larger
capacity
leads to best
accuracies

Fits the MCU on-chip
memory capacity if
quantized to 8 bit

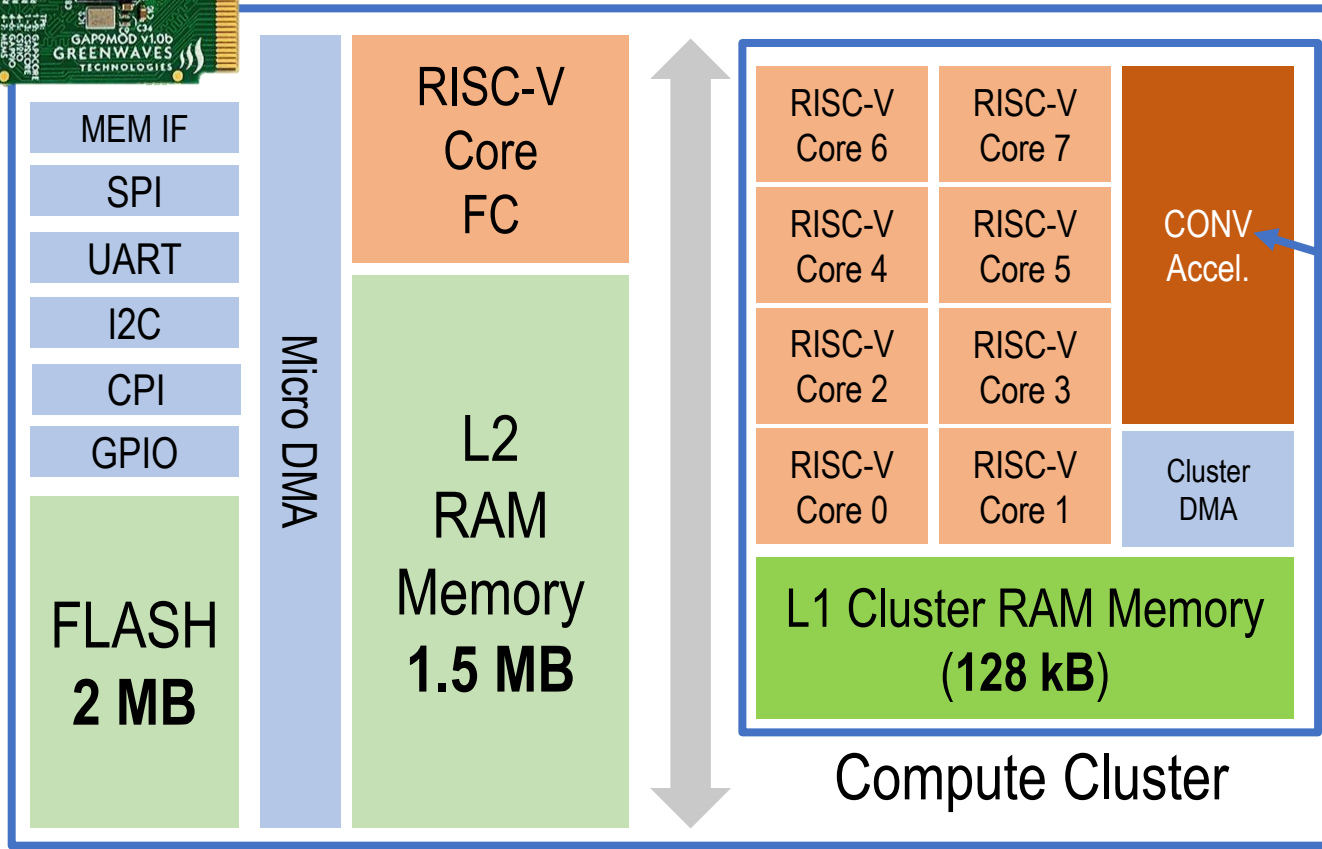
More examples improves
overall recognition

Processors for Extreme Edge Devices



- OPS refers to peak MAC operations/s at the minimum available bitwidth. Perf from specialized accelerator not accounted.
- Estimates from available numbers may be inaccurate. Feedbacks are welcome!

GAP9 Microcontroller



- Compute Cluster for AI/ML workload
 - General purpose RISC-V cores W/ vectorized INT8/FP16 MACs
 - **Convolution HW Accelerator for 1x1 and 3x3 operators**
- 25mW at 240MHz @ 0.65V

https://greenwaves-technologies.com/gap9_processor/

On-Device Analysis (GAP9)

Feature Extractor	Performance 10 shots w/ 4+2 way		DNN Inf Time	Eff.	Energy per inf
	Full-Precision	Quantized 8-bit	msec	MAC/cyc	μJ
TCRES8	ACC: 0.62 FAR: 0.0083	ACC: 0.63 FAR: 0.0079	1.11	9.1	28.3
DSCNN S	ACC: 0.69 FAR: 0.0088	ACC: 0.69 FAR: 0.0085	0.61	18.3	35.7
DSCNN M	ACC: 0.66 FAR: 0.0064	ACC: 0.66 FAR: 0.0061	1.7	22.3	94.8
DSCNN L	ACC: 0.70 FAR: 0.0049	ACC: 0.69 FAR: 0.0051	4.08	27.7	186.2
RES15	ACC: 0.73 FAR: 0.0013	ACC: 0.73 FAR: 0.0013	9.21	66.8	246.9

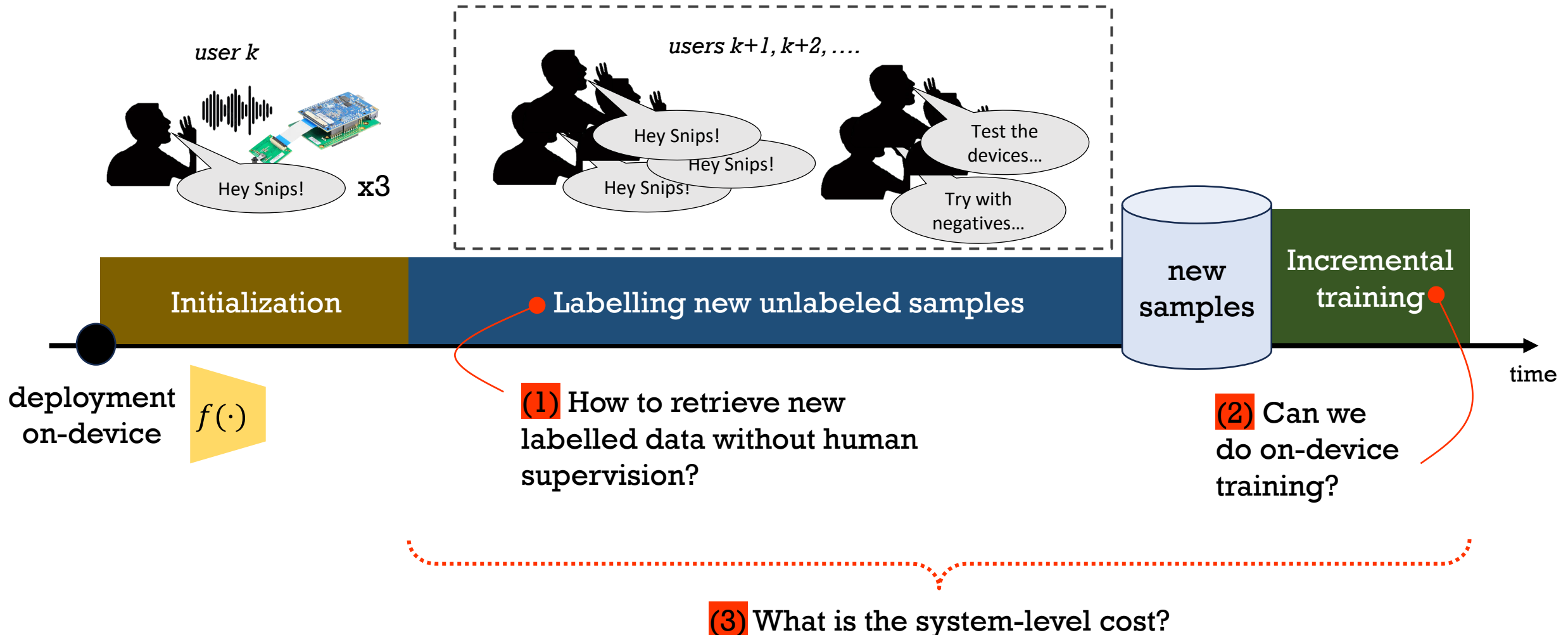
Accuracy upper-bounded by the frozen model

<1/10 latency constraint

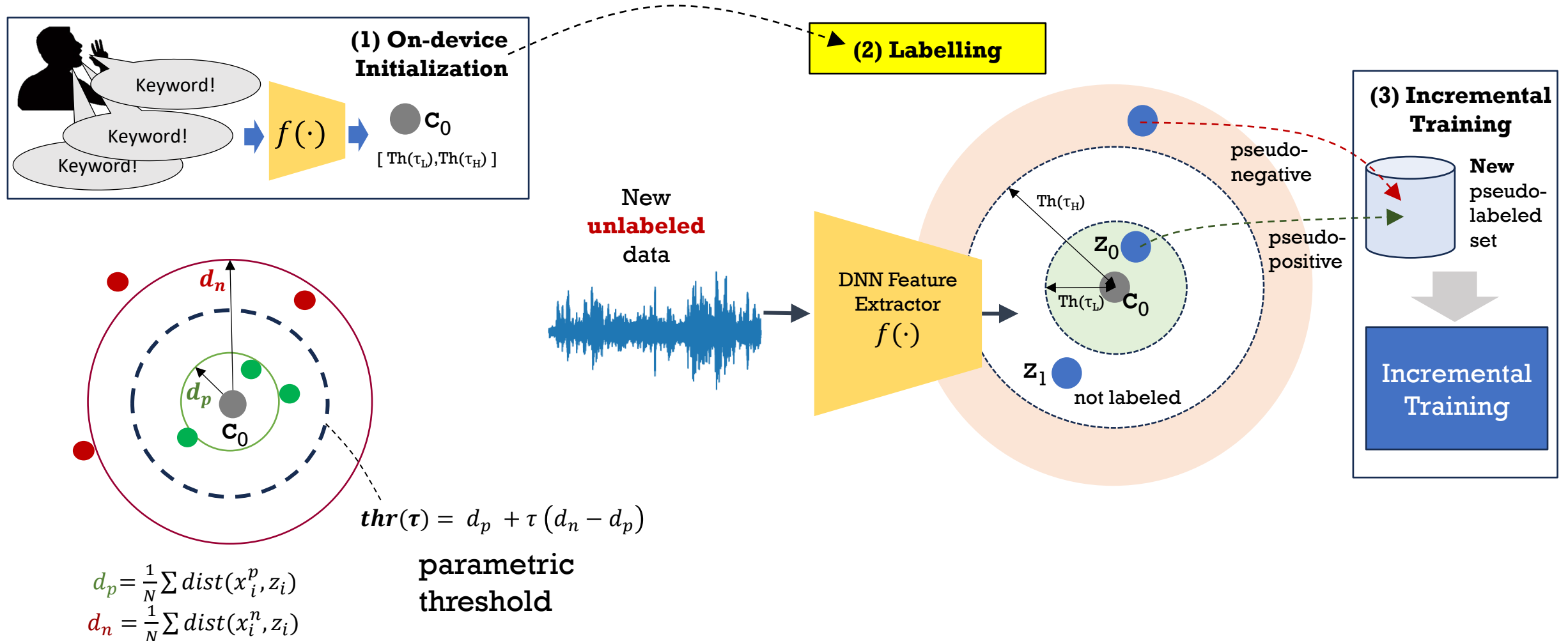
$$iS_{kwd} = d_{L_2}(f(x), c_j) < \gamma = \text{margin}$$

- **Almost lossless** 8-bit post-training quantization using GAP9's toolset
- dominated by 1x1 conv with peak eff of 40 MAC/cyc (~4x than 8 cores)
- 2.5x faster than DSCNN-L irrespective of the 8.4x more MAC
- 3x3 conv faster than 1x1, with peak efficiency of 100 MAC/cyc

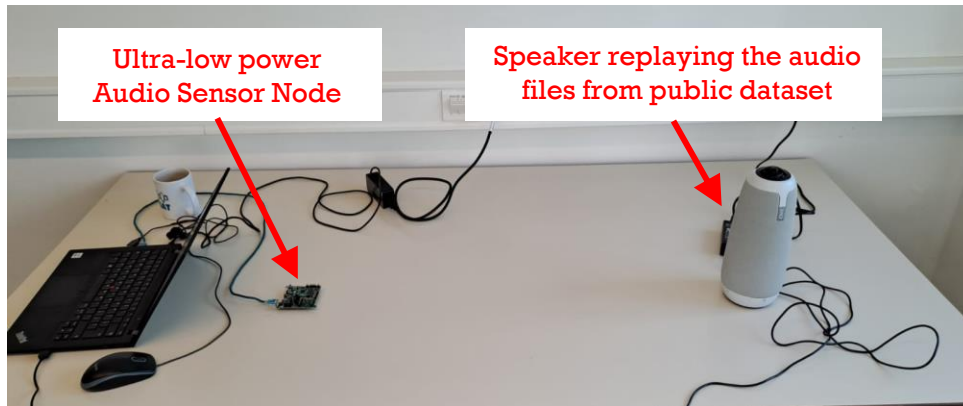
“Practical” Self-Training for Personalized KWS



Labelling new data for Self-Training



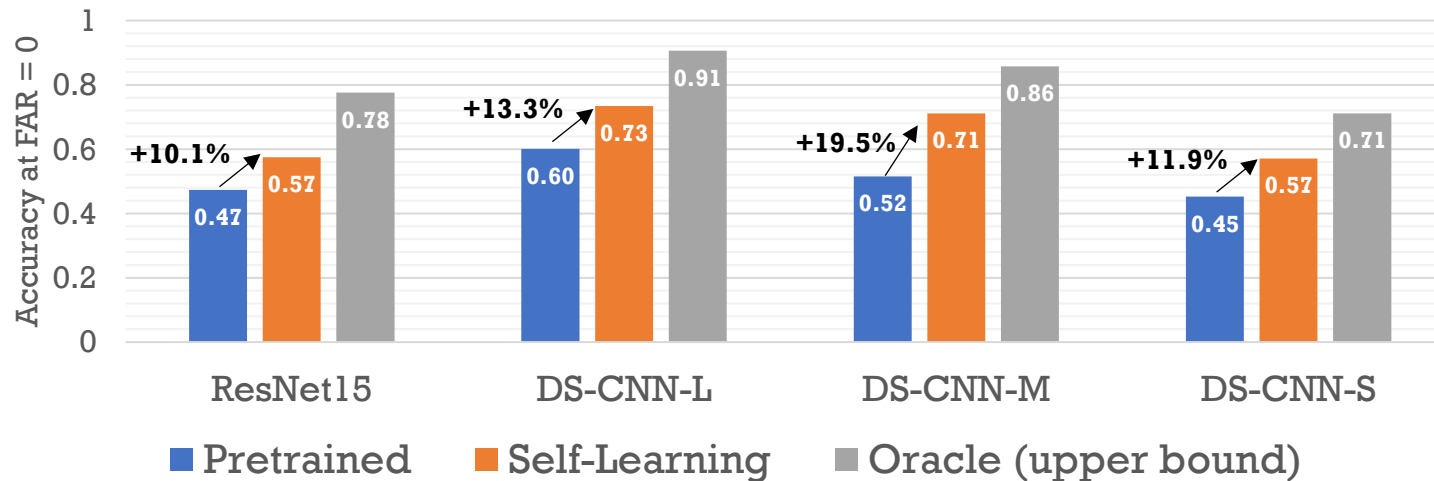
User-Personalized KWS



New “HeySnips” keyword

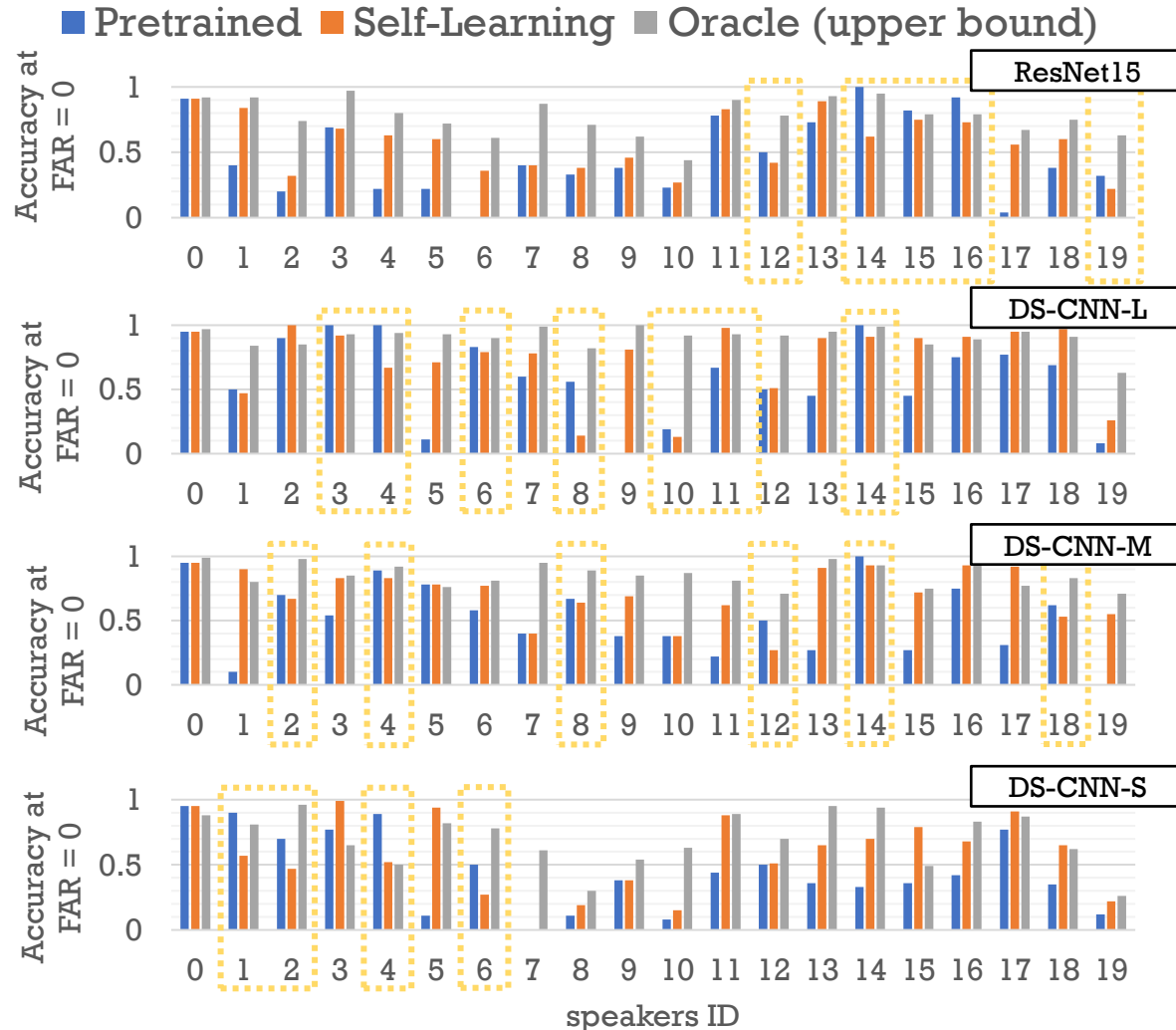
Recorded dataset

- Test
 - 20 speakers x 10/11 samples (3-shot)
 - 150 negatives
- Train/adapt
 - 400 samples (15.75% pos), label removed



Accuracy Improvements w/ only new unlabeled data

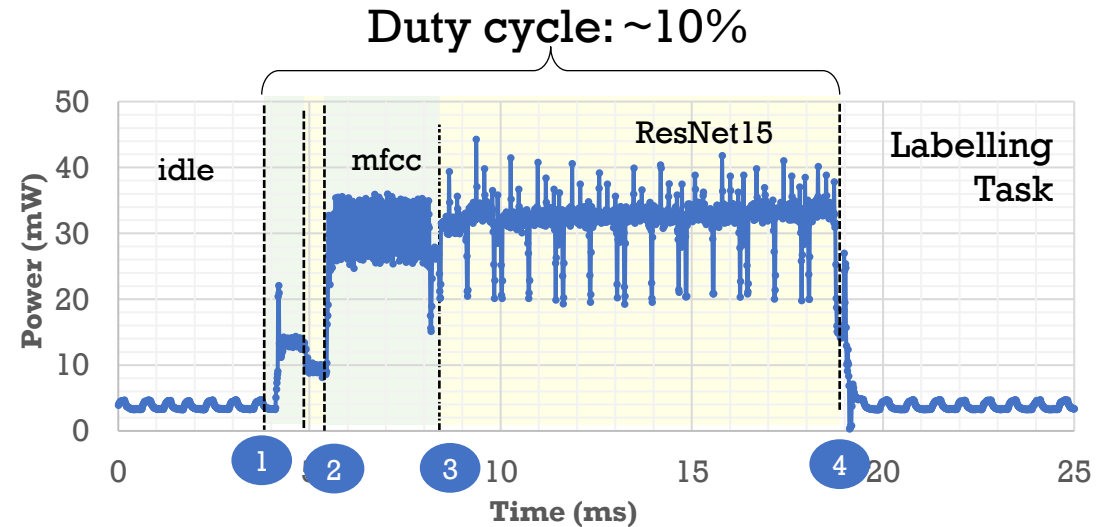
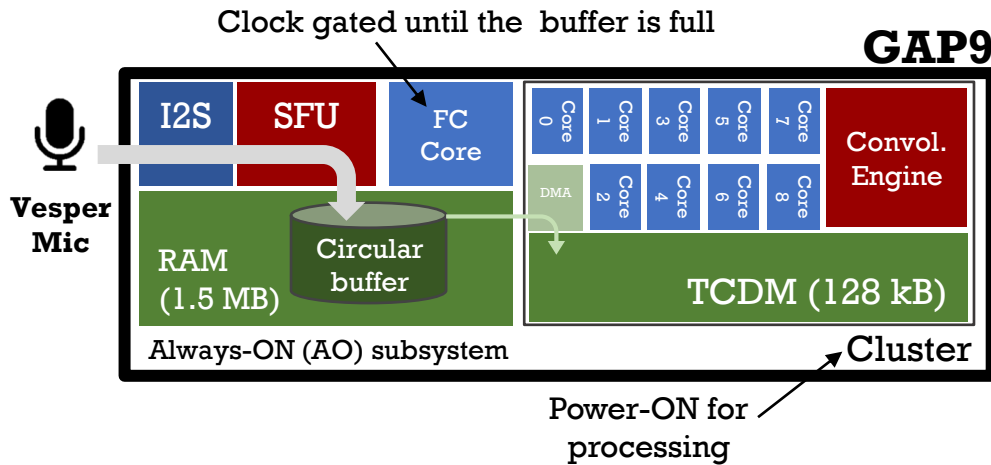
Self-Training is not a free lunch



- For some users, self-training underperforms w.r.t. pretrained model
- More visible for low-data experiments than larger public data.
- More investigation is needed!
 - Low-data regime
 - Forgetting (?)
 - Mechanisms to detect the divergence and prevent update

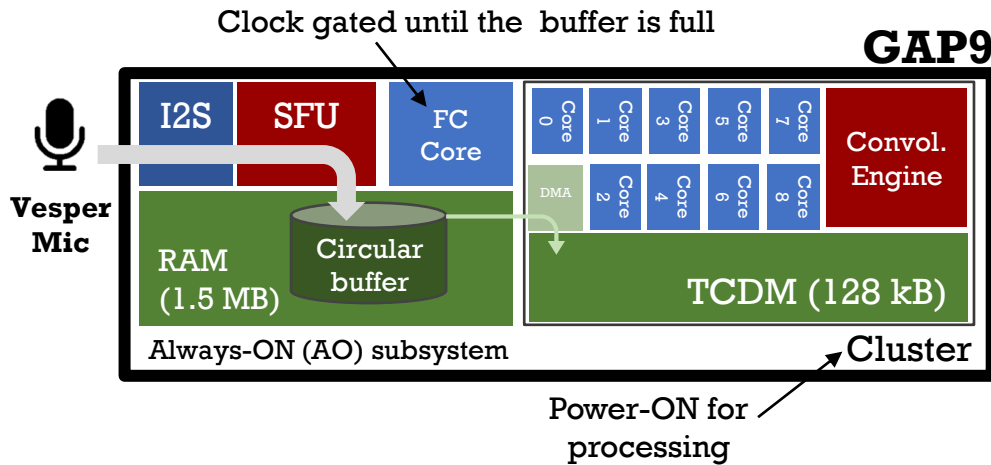
pretrained > self-training

System Level Power Management

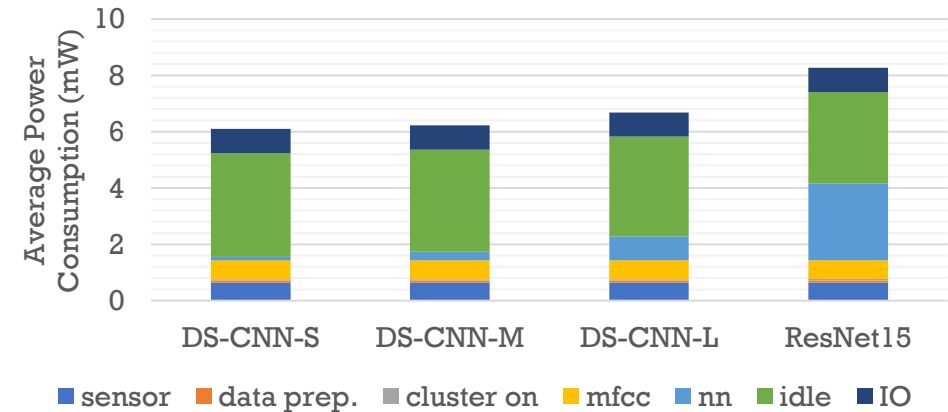
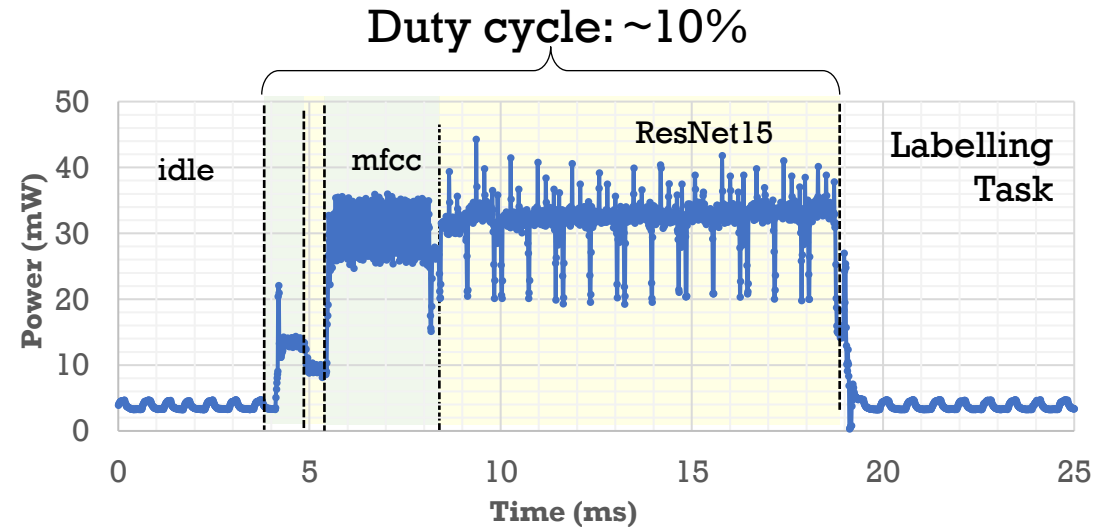


```
function labeling_task() {
    // after setup
    while (True):
        1 Wait SFU event          // exit from idle state
          Set FC clk to 240MHz
          Data Preparation (cast, zero mean)
          Turn ON Cluster
        2 Parallel MFCC Processing on Cluster
        3 Parallel NN Processing On Cluster
        4 Turn OFF Cluster
          Pseudo-label assignment
          Set FC clk to 1 MHz // enter idle state
    }
```

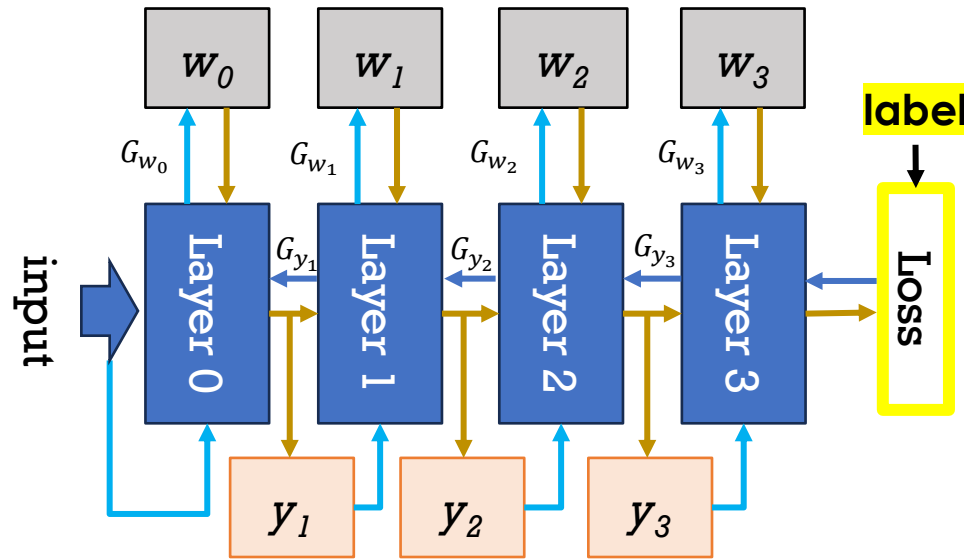
System Level Power Management



➤ processing costs become hidden w.r.t. the acquisition cost (idle)



And the training?



Convolution Layer i

(batch)

Forward (FW)

$$y_{i+1} = w_i \cdot y_i$$

+ storing y_i

Note:

$$G_x = \frac{\partial L}{\partial x}$$

Backward input gradient (BW_{ig})

$$G_{y_i} = w_i \cdot G_{y_{i+1}}$$

Backward weight gradient (BW_{wg})

$$G_{w_i} = y_i \cdot G_{y_{i+1}}$$

then, update
 $w_i \leftarrow w_i + \eta G_{w_i}$



```
pulp_linear_fp32_fw_cl(&lin_args);
```

```
pulp_CrossEntropyLoss(&loss_args);
```

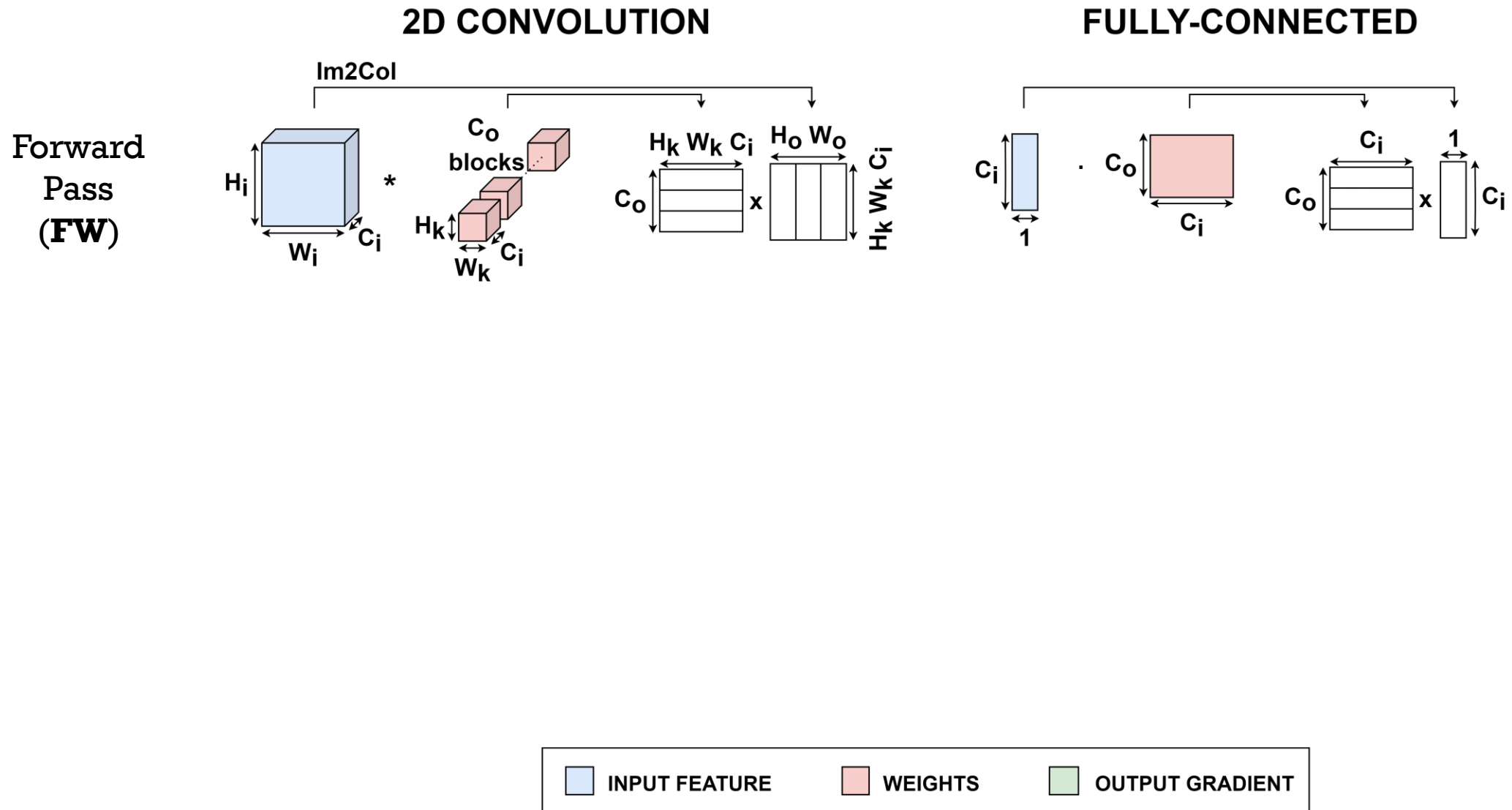
```
pulp_linear_fp32_bw_input_grads_cl(&lin_args);  
pulp_linear_fp32_bw_param_grads_cl(&lin_args);
```

```
pulp_gradient_descent_fp32(&wgts);
```

The first **latency-optimized open-source training library for RISC-V multi-core MCUs**

>40x MAC/cyc than state-of-the-art

From Conv To MatMul Kernels

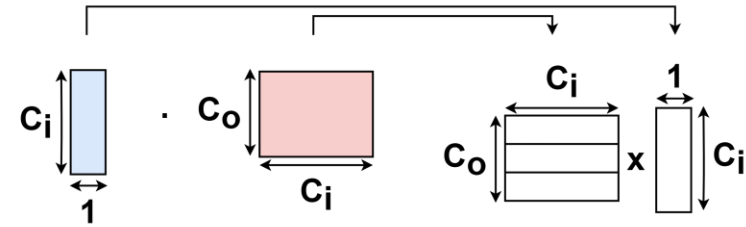
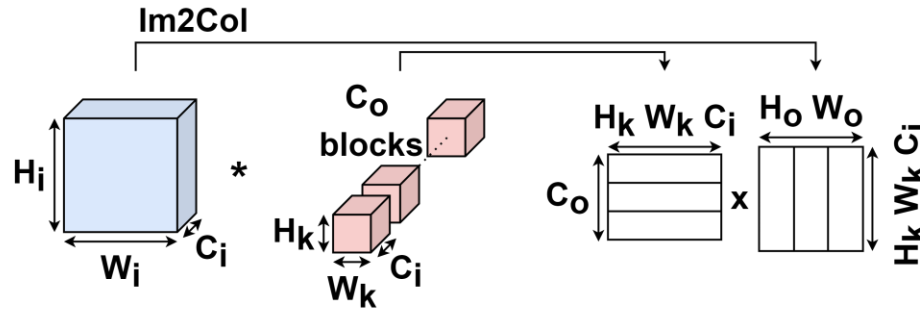


From Conv To MatMul Kernels

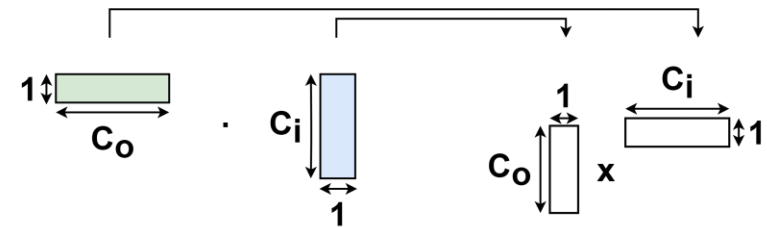
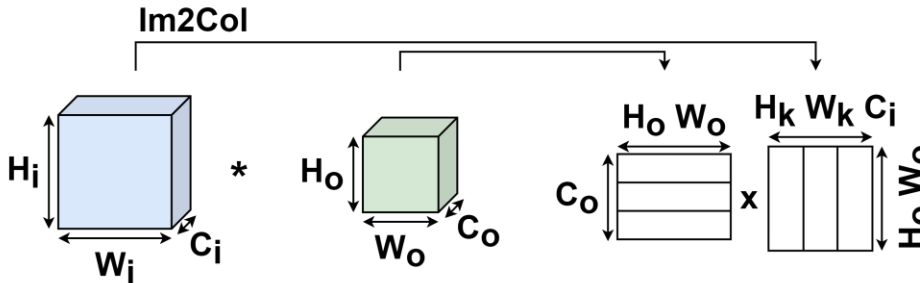
2D CONVOLUTION

FULLY-CONNECTED

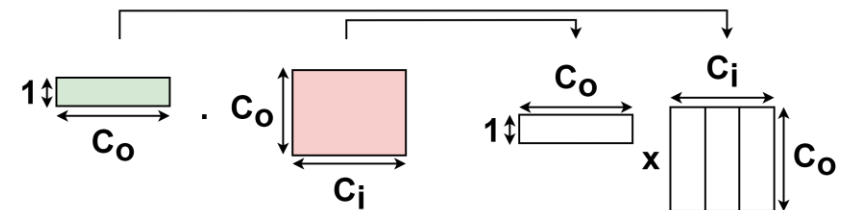
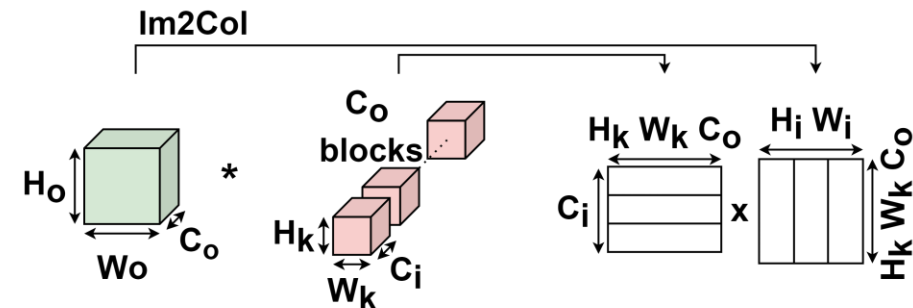
Forward
Pass
(**FW**)



Weight-Grad
Backward
(**WG-BW**)

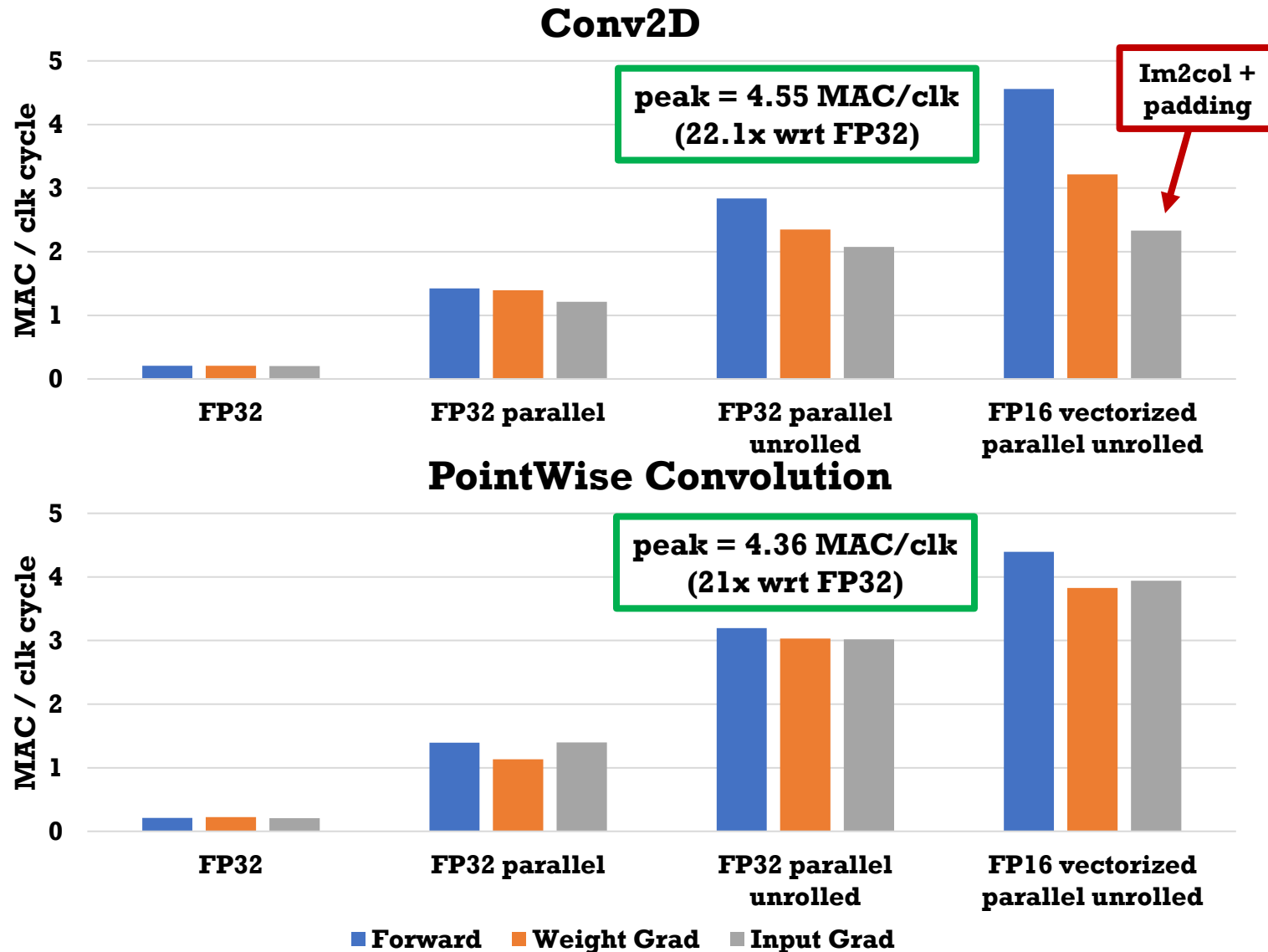


Activation-Grad
Backward
(**IG-BW**)



Training MAC =
3x Inference MAC

PULP-TrainLib: Acceleration



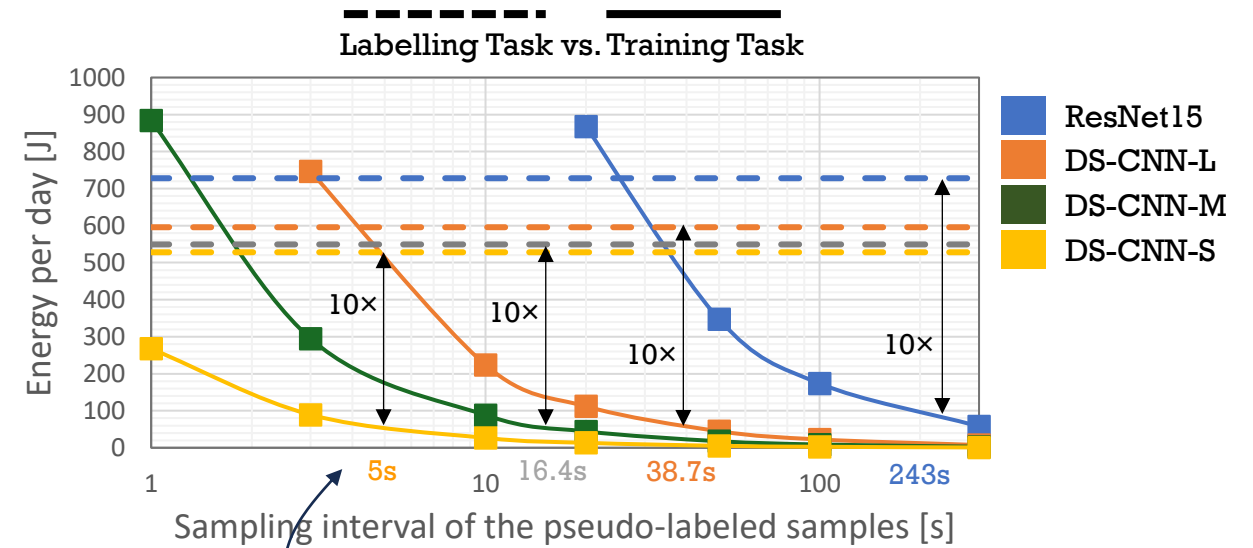
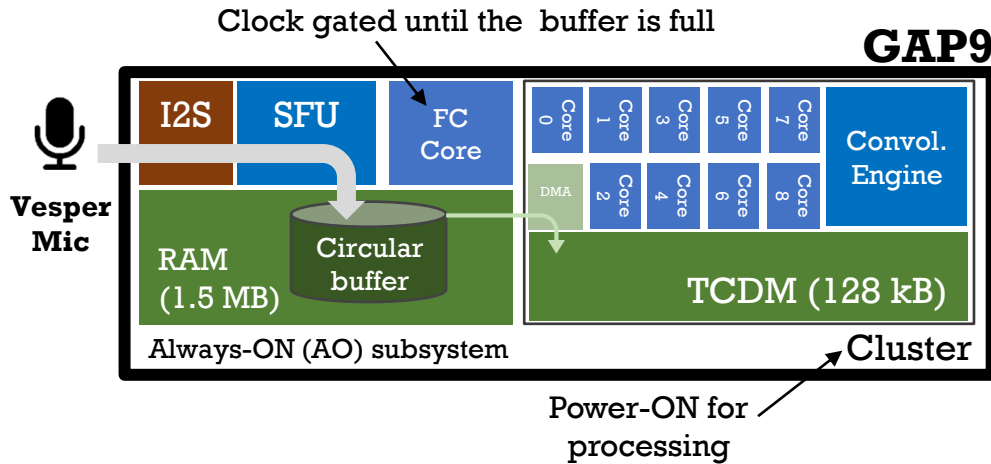
OPTIMIZATIONS:

- FP32 Parallelization
- FP32 Unrolling + Parallelization
- FP16 Vectorization + Unrolling + Parallelization

SUPPORTED FEATURES:

- **Layers:** Conv2D, PointWise, DepthWise, Linear
- **Activations:** ReLU, SoftMax
- **Losses:** MSE, CrossEntropy
- **Optimizers:** SGD
- **Pooling:** MaxPool, AvgPool

System Level Evaluation



on-device training energy **10× lower** than the labeling energy if sampling a new utterance every 5sec (DS-CNN-S)

On-device Learning perf traded vs. inference/labeling task

Where We Are and What's Next

Supported by:



- HW for low-power embedded AI is here (not yet a free lunch ☺)
 - Specialized HW + Optimized SW + Ad-hoc trained DNN models
 - Effective w/ new architectures
 - Automate the data-to-HW design optimization tools
- Next: **towards adaptive extreme-edge solutions**
 - focus on real-world robustness/adaptation over time rather than solving datasets
 - HW/SW for on-device adaptation, aka continual learning
 - Heterogenous compute units is key (inference-learning-inference-....)
 - Benchmarks beyond classical static dataset (and more real-world data)
 - Tools to automate on-device learning (data precision, batching, new learning paradigms...)
 - Reliability and control of self-learning still an open question

Thanks for the attention

manuele.rusci@esat.kuleuven.be

